Laboratorio N° 3

"Tema: Diseño de una ISA a partir de la ISA LC3"

FACULTAD DE INGENIERÍA - UNAM

CARRERA:

Ingeniería en Computación

MATERIAS:

IC221 - Arquitectura de computadoras

DOCENTES:

ING. CARBALLO, Roberto E.

ING. BENITEZ, Aldo J.

ING. SCHUSTER, Roberto N.

ING. ROSSLER, Gustavo O.

ING. Diaz, Eduardo.

SRTA. ANDREOLI, Ainara Belén

SRTA. LOPEZ, Alejandra B.

SR. CRUZ, Thiago A.

SR. ARANDA, Brian E.

ALUMNOS:

- Arce, Andrés Agustin
- Becker, Ramón Angel David
- Cuadra, Josias Ramon

GRUPO: 22

ÍNDICE

1.1 DESARROLLO 4
1.1.1 Implementación Inicial y Diagnóstico con IA : 4
2 Decisiones Técnicas 6
2.1 DESARROLLO 6
2.1.1 Implementación: Operaciones de conversión y ajuste de escala 7
3 Diseño del Algoritmo y Pruebas 8
3.1 DESARROLLO 8
3.1.1 Implementación: Pruebas y Validación de Resultados 11
4 Implementación en ASSEMBLY 15
4.1.1 Implementación: Rutinas en LC-3 y Validaciones 16
5 Segunda Parte: Cálculo de Seno y Coseno por Tabla 18
5.1.1 Planteo Metodológico: 18
5.2 Decisiones Técnicas y Ajustes de Arquitectura: 19
5. 3 Diseño del Algoritmo y Optimización del Dominio: 19
5.3.1 Dilema Algorítmico: Series de Taylor frente a Tablas de Consulta 20
5.3.2 Cálculo Intermedio (Promedio vs. Interpolación) 22
6 Implementación y Pruebas Unitarias en C++ 23
6.1 El Polinomio y el Giro de 100 a 91 Muestras (1° por Entrada) 24
6.2 ¿Tabla Bidimensional (X, Y) o Vector Único? 25
6.3 Errores en la Generación de .FILL y el Caso Borde de 16384 25
6.4 Lógica de Consulta: Formatos, Cuadrantes e Índices 26
6.5 Clasificador de Cuadrantes: 27
Sustitución de CMP y SUB mediante Aritmética en Complemento a Dos 27
6.6 "Casos Especiales" sobre los Ejes Cartesianos 28
6.7 Evolución del Prototipo en C++: 29
6.8 Incorporación del Cálculo de Índice y Ángulo de Referencia 30
6.9 Cuatro Cuadrantes y Cinco Casos Especiales 31
7 Estudio Comparativo para el Cálculo Entero del Índice 32
7.1 Resultados del Barrido Exhaustivo, Auditoría del Caso Crítico (4754) y Criterio de Decisión 33
7.2 Integración Modular e Interpolación Lineal Continua 34
Implementación y Validación Exhaustiva de la Interpolación Lineal 35
7.3 Función (calcular_seno_coseno_Q1_14) y Unificación de Casos Especiales 36
7.4 Prueba Integral del Ciclo Completo (0 a 2) 37
Evaluación de printf() frente a cout 37
8 Planificación Arquitectónica para la Traducción a Assembly LC-3 38
8.1 Entorno de Pruebas y Estrategia de Desarrollo en Assembly 38
8.2 Prueba 01: Carga de un Elemento de la Tabla 39
8.3 Prueba 02: Acceso Dinámico a la Tabla 39
8.5 Prueba 03: Reserva y Verificación de Variables Temporales 40
8.6 Formato de la Salida: la Tabla ya Entrega Q1.14 41
8.7 Prueba 04: Determinación del Cuadrante 42
8.8 Prueba 05: Detección de Casos Especiales 43
8.9 Optimización: Aprovechamiento de los Códigos de Condición 43
8.10 Prueba 06: Ángulo de Referencia y Cofunción 44
8.11 Prueba 07: Determinación y Almacenamiento de los Signos 45
8.12 Prueba 08 A: Conversión de la Referencia a Grados Q8.8 e Índice de la Tabla 47
8.13 Prueba 08 B: Índices de Seno y Coseno a partir de la Cofunción 48
8.14 Prueba 08 C: Consulta de los Valores en la Tabla 50
8.15 Prueba 08 D: Aplicación de los Signos 51
8.16 Prueba 08 E: Integración con la Determinación de Cuadrantes y Entrada Única 52
9 Conclusión y Trabajo Futuro 53
10 REFERENCIAS BIBLIOGRÁFICAS 54

1. Primera Parte : Punto Fijo

1.1 DESARROLLO

En esta primer etapa del Laboratorio 3, se tuvo como objetivo general el comprender y aplicar la representación numérica en formatos de punto fijo dentro de un procesador, buscando desarrollar a nivel criterioso y técnico el alcance y limitaciones de la Inteligencia Artificial Generativa como herramienta de apoyo tanto para un área de aprendizaje como de escritura de código, complementando en lenguajes de alto nivel y en lenguaje de ensamblador (Assembly).

1.1.1 Implementación Inicial y Diagnóstico con IA :

En una primera instancia, se desarrolló de forma manual un código base en C++ orientado a comprender la mecánica del formato Q sobre palabras de 16 bits. Donde este primer prototipo permite ingresar dos números en punto flotante (float), convertirlos respectivamente a los formatos Q3.12 y Q7.8 realizar un casting (valor * escala + 0.5f), e imprimir su representación en decimal, hexadecimal y binario ubicando visualmente el punto fijo. Incluyendo también una aproximación a la compatibilización de escalas realizando un desplazamiento de 4 bits a la derecha para alinear ambos valores en Q7.8 y efectuar su suma.

Con este código preliminar, se recurre a un modelo de Inteligencia Artificial Generativa donde se le suministró el listado en C++ y lo que requería la consigna, con el objetivo de suministrar un grado de cumplimiento del programa y de esta forma identificar puntos ciegos.

Gracias a esto se obtuvo dos conclusiones fundamentales para guiar el desarrollo del primer punto:

- Alcance: Aunque la actividad iba bien encaminada tanto en la lectura, casting o alineación para la suma, el programa aún estaba incompleto para operaciones de resta, multiplicación y división.
- Casos límite: Se llegó a un análisis conjunto donde se advirtió que el método empleado para realizar el redondeo durante el casting y el uso del operador de desplazamiento presentaban limitaciones al momento de operar con números negativos, además de que ante un caso de desbordamiento no contaba con los controles para evitarlos.

Con este diagnóstico se estructuró un plan de trabajo donde antes de escribir algún tipo de código definitivo se tomaron en cuenta una serie de criterios y decisiones técnicas para corregir estas limitaciones.

```cpp
#include <iostream>
#include <stdint.h>
#include <stdio.h>
#include <bitset> // Para imprimir binario de 16 bits
using namespace std;

// ! Funcion unicamente demostrativa de como quedaria el formato Q resultante
void binario(int16_t num, char salida[18], int Q_entera) {
// Necesitamos un arreglo de al menos 18 chars (16 bits + 1 punto + 1 '\0')
    int idx = 0;
    for (int bit = 15; bit >= 0; bit--) {
        salida[idx++] = ((num >> bit) & 1) ? '1' : '0';

        // El punto va justo despues de haber impreso el
        // ultimo bit entero (15 - Q entera)
        if (bit == (15 - Q_entera)) {
            salida[idx++] = '.';
        }
    }
    salida[idx] = '\0';
}

int main() {
    float num1, num2;
    int16_t num1_Q3_12, num2_Q7_8, num1_Q7_8, suma_Q7_8;
    char bin1[18], bin2[18], bin_suma[18];

    cout << "Ingrese el 1er numero (para Q3.12): "; cin >> num1;
    cout << "Ingrese el 2do numero (para Q7.8): "; cin >> num2;
    num1_Q3_12 = (int16_t)(num1 * (1 << 12) + 0.5f);
    num2_Q7_8  = (int16_t)(num2 * (1 << 8) + 0.5f);
    binario(num1_Q3_12,bin1, 3);
    binario(num2_Q7_8,bin2, 7);

    cout << "\n--- FORMATOS Q INICIALES ---" << endl;
    printf("num1 en Q3.12: Dec = %-6d | Hex = x%04X | Bin = ", num1_Q3_12, (uint16_t)num1_Q3_12);
    cout << bin1 << endl;

    printf("num2 en Q7.8:  Dec = %-6d | Hex = x%04X | Bin = ", num2_Q7_8, (uint16_t)num2_Q7_8);
    cout << bin2 << endl;

    cout << "\n_____________________________________________________\n" << endl;

    // Alineacion a Q7.8 (menor f)
    num1_Q7_8 = num1_Q3_12 >> 4;
    suma_Q7_8 = num1_Q7_8 + num2_Q7_8;
    binario(num1_Q7_8,bin1,7);
    binario(suma_Q7_8,bin_suma,7);

    cout << "--- ALINEACION Y SUMA EN Q7.8 ---" << endl;
    printf("num1 formateado a Q7.8: Dec = %-6d | Hex = x%04X | Bin = ", num1_Q7_8, (uint16_t)num1_Q7_8);
    cout << bin1 << endl;

    printf("Suma final en Q7.8:     Dec = %-6d | Hex = x%04X | Bin = ", suma_Q7_8, (uint16_t)suma_Q7_8);
    cout << bin_suma << endl;

    return 0;
}
```

Figura 1 - Primer Código realizado

2 Decisiones Técnicas

2.1 DESARROLLO

Durante esta fase de correcciones no se aceptaron las sugerencias del modelo de manera automática, sino que se evaluó cada propuesta pensando en el funcionamiento interno del procesador y en su posterior traducción al lenguaje ensamblador de la LC-3:

- Descarte de librería: Al detectar que la expresión original redondeaba de forma errónea los números negativos la IA propuso incluir la librería <cmath> y utilizar lroundf(). Se tomó la decisión técnica de rechazar esta función ya que el emplear rutinas de librería ocultaba la lógica algorítmica para programar luego en Assembly. Por eso en su lugar se exigió resolver este redondeo mediante condicionales **if / else**, donde se le sumaría **0.5** si el número fuera **positivo** y restando **0.5** si es **negativo**.
- Formato de salida en Q7.8: Para las operaciones de suma, resta, multiplicación y división se decidió adoptar **Q7.8** como formato destino común. Mientras que el formato **Q3.12** ofrecía una alta precisión fraccionaria más un rango entero muy estrecho, el formato **Q7.8** brindaba un rango mucho mayor lo que reducía drásticamente la probabilidad de desbordamiento al momento de operar.

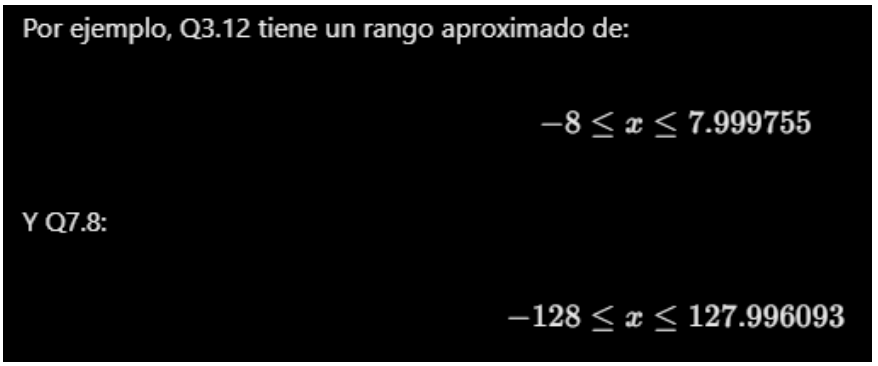


Figura 2 - Precisión de cada Rango

- Variables Intermedias de 32 bits y Overflow: Operar directamente en **16 bits** durante una multiplicación o un pre-escalado de división produce desbordamientos inmediatos. Por lo que se decidió realizar los cálculos intermedios en registros de **32 bits** y validar explícitamente que el resultado reescalado se encuentre dentro del intervalo antes de realizar el casting final a 16 bits.

2.1.1 Implementación: Operaciones de conversión y ajuste de escala

A continuación se detallan y justifican las operaciones matemáticas implementadas para resolver estos problemas:

- Suma y resta: Para alinear el punto binario del primer operando con el segundo, es necesario reducir de 12 a 8 la cantidad de bits fraccionarios. En lugar de aplicar un truncamiento directo por desplazamiento realiza una división entera por 2^4 incorporando una rutina de redondeo manual, evitando perder precisión innecesariamente antes de efectuar la suma y resta.

$$Q7.8 = \frac{Q3.12}{2^4}$$

Figura 3 - Conversión Q3.12 a Q7.8

- Multiplicación y ajuste del formato Q: Al multiplicar el entero A (Q3.12) por el entero B (Q7.8), las escalas se combinan generando un producto intermedio de 32 bits en formato Q10.20. Para expresar su resultado final en Q7.8, se descartan los 12 bits fraccionarios sobrantes dividiendo el producto entero por 2^{12} con redondeo.

$$Q3.12 \times Q7.8 = Q10.20$$

$$Resultado = \frac{A \times B}{2^{12}}$$

Figura 4 - Operación para Multiplicación

- División y conservación fraccionaria: Partiendo de la relación entre los operandos escalados, una división entera entre A y B dejaría el cociente en escala Q11.4, perdiendo resolución fraccionaria. Siendo la opción correcta para obtener directamente el resultado en Q7.8 una pre-escala del numerador multiplicando por 16 en una variable temporal de 32 bits antes de dividir por B.

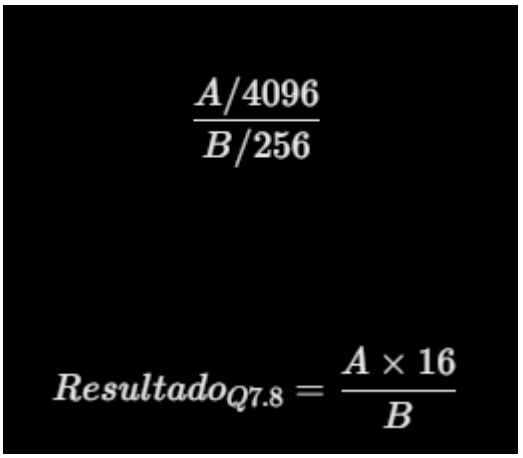


Figura 5 - Operacion para División

3 Diseño del Algoritmo y Pruebas

3.1 DESARROLLO

A partir de las decisiones técnicas establecidas anteriormente, se estructuró el programa definitivo el lenguaje de alto nivel, dividiendo el diseño en funciones auxiliares explícitas, y un flujo principal secuencial que lo convierte en una estructura fácil de analizar.

- Validar Entrada : Calcula los límites mínimos y máximos admitidos por un entero con signo de 16 bits en el formato Q indicado, rechazando entradas que producirían un casting inválido antes de operar.

```
bool validar_entrada(float valor, int bits_fraccionarios) {

    float escala = 1 << bits_fraccionarios;

    float minimo = -32768.0f / escala;
    float maximo = 32767.0f / escala;

    if (valor < minimo || valor > maximo) {
        return false;
    }

    return true;
}
```

Figura 6 - Validación de entrada

- Convertir a Q: Se encarga de escalar el número flotante multiplicando por 2^n y aplica redondeo simétrico manual antes de truncar mediante el casting al entero.

```
int16_t convertir_a_Q(float valor, int bits_fraccionarios) {

    float escala = 1 << bits_fraccionarios;
    float temporal = valor * escala;

    if (temporal >= 0) {
        temporal = temporal + 0.5f;
    } else {
        temporal = temporal - 0.5f;
    }

    return (int16_t)temporal;
}
```

Figura 7 - Conversión a Q

- División redondeada: Resuelve tanto la reducción de bits fraccionarios como el cociente de la división en punto fijo. Separando los signos de los operadores, calculando el cociente y el resto en magnitud e incrementando el cociente si el doble del resto iguala o supera al divisor.

```
int32_t division_redondeada(int32_t numerador, int32_t denominador) {

    if (denominador == 0) {
        return 0;
    }

    int signo = 1;

    if (numerador < 0) {
        signo = -signo;
        numerador = -numerador;
    }

    if (denominador < 0) {
        signo = -signo;
        denominador = -denominador;
    }

    int32_t cociente = numerador / denominador;
    int32_t resto = numerador % denominador;

    if (resto * 2 >= denominador) {
        cociente++;
    }

    return cociente * signo;
}
```

Figura 8 - Redondeo de división

- Dentro de Rango: Se encarga de verificar que cualquier resultado intermedio calculado en 32 bits se mantenga dentro del rango representable de 16 bits antes de almacenarlo en su variable definitiva.

```
bool dentro_de_rango(int32_t valor) {
    return (valor >= -32768 && valor <= 32767);
}
```

Figura 9 - Verificación de rango

- Binario y Mostrar Q: Son funciones de visualización las cuales extraen bit a bit la representación en complemento a 2 de 16 bits e insertan el punto binario según la cantidad de bits de la parte entera, mostrando en pantalla el valor decimal escalado, su código hexadecimal y su cadena binaria.

```
void binario(int16_t num, char salida[18], int Q_entera) {

    uint16_t bits = (uint16_t)num;
    int idx = 0;

    for (int bit = 15; bit >= 0; bit--) {

        salida[idx++] = ((bits >> bit) & 1) ? '1' : '0';

        if (bit == (15 - Q_entera)) {
            salida[idx++] = '.';
        }
    }

    salida[idx] = '\0';
}

void mostrar_Q(const char* nombre, int16_t valor, int Q_entera) {

    char representacion[18];

    binario(valor, representacion, Q_entera);

    cout << nombre << endl;

    printf("Decimal: %d | Hexadecimal: x%04X | Binario: %s\n",
           valor, (uint16_t)valor, representacion);
}
```

Figura 10 - Muestra

A continuación se estará mostrando la parte del programa principal :

```cpp
int main() {

    float num1, num2;

    int16_t num1_Q3_12;
    int16_t num2_Q7_8;

    int16_t num1_Q7_8;
    int16_t suma_Q7_8;

    int32_t suma_temporal;
    int32_t producto_temporal;
    int32_t producto_Q7_8;

    int32_t division_temporal;
    int32_t division_Q7_8;

    // ------------------------------------------------
    // 1. INGRESO DE NUMEROS
    // ------------------------------------------------

    cout << "Ingrese el primer numero (Q3.12): ";
    cin >> num1;

    cout << "Ingrese el segundo numero (Q7.8): ";
    cin >> num2;

    // ------------------------------------------------
    // 2. VALIDACION DE ENTRADAS
    // ------------------------------------------------

    if (!validar_entrada(num1, 12)) {
        cout << "\nERROR: el primer numero esta fuera del rango Q3.12.\n";
        return 0;
    }

    if (!validar_entrada(num2, 8)) {
        cout << "\nERROR: el segundo numero esta fuera del rango Q7.8.\n";
        return 0;
    }

    // ------------------------------------------------
    // 3. CONVERSION A FORMATOS Q DIFERENTES
    // ------------------------------------------------

    num1_Q3_12 = convertir_a_Q(num1, 12);
    num2_Q7_8  = convertir_a_Q(num2, 8);

    cout << "\n===== FORMATOS Q INICIALES =====\n";

    mostrar_Q("Numero 1 - Q3.12:", num1_Q3_12, 3);
    cout << endl;

    mostrar_Q("Numero 2 - Q7.8:", num2_Q7_8, 7);
    cout << endl;

    // ------------------------------------------------
    // 4. CONVERSION DE Q3.12 A Q7.8 Y SUMA
    // ------------------------------------------------

    cout << "\n===== ALINEACION Y SUMA =====\n";

    num1_Q7_8 = (int16_t)division_redondeada(num1_Q3_12, 16);

    mostrar_Q("Numero 1 convertido a Q7.8:", num1_Q7_8, 7);
    cout << endl;

    suma_temporal = (int32_t)num1_Q7_8 + num2_Q7_8;

    if (dentro_de_rango(suma_temporal)) {

        suma_Q7_8 = (int16_t)suma_temporal;
        mostrar_Q("Suma final en Q7.8:", suma_Q7_8, 7);

    } else {
        cout << "ERROR: desbordamiento en la suma.\n";
    }

    cout << endl;

    // ------------------------------------------------
    // 5. MULTIPLICACION EN PUNTO FIJO
    // ------------------------------------------------

    cout << "\n===== MULTIPLICACION =====\n";

    producto_temporal = (int32_t)num1_Q3_12 * (int32_t)num2_Q7_8;

    cout << "Producto intermedio (escala Q10.20): "
         << producto_temporal << endl;

    producto_Q7_8 = division_redondeada(producto_temporal, 4096);

    cout << endl;

    if (dentro_de_rango(producto_Q7_8)) {

        mostrar_Q("Producto final en Q7.8:",
                  (int16_t)producto_Q7_8, 7);

    } else {
        cout << "ERROR: desbordamiento en la multiplicacion.\n";
    }

    cout << endl;

    // ------------------------------------------------
    // 6. DIVISION EN PUNTO FIJO
    // ------------------------------------------------

    cout << "\n===== DIVISION =====\n";

    if (num2_Q7_8 == 0) {

        cout << "ERROR: no se puede dividir por cero.\n";

    } else {

        division_temporal = (int32_t)num1_Q3_12 * 16;

        division_Q7_8 = division_redondeada(
            division_temporal, num2_Q7_8
        );

        if (dentro_de_rango(division_Q7_8)) {

            mostrar_Q("Division final en Q7.8:",
                      (int16_t)division_Q7_8, 7);

        } else {
            cout << "ERROR: desbordamiento en la division.\n";
        }
    }

    cout << "\n===== FIN DEL PROGRAMA =====\n";

    return 0;
}
```

Figura 11 - Parte MAIN del programa

3.1.1 Implementación: Pruebas y Validación de Resultados

Para comprobar la solidez del algoritmo antes de traducir la lógica al lenguaje ensamblador de la LC-3, se diseñó un plan de pruebas progresivo destinado a evaluar operaciones básicas, manejo de signos en complemento a 2, fracciones puras, y casos de desbordamiento.

Número 1	Número 2	Qué comprobamos
1.5	2	Operaciones básicas
-1.5	2	Números negativos
0.5	0.5	Resultados fraccionarios
0	5	Multiplicación y división por cero como numerador
1.2345	2.3456	Pérdida de precisión
7.9	120	Límites de representación

Figura 12 - Planificación de pruebas a realizar

- Verificación del Caso Base: Se evaluaron los valores de entrada 1.5 (formato Q3.12 = 6144) y 2 (formato Q7.8 = 512). Al tratarse de números cuya parte fraccionaria era potencia exacta de 2, la conversión no presenta errores de redondeo y todas las operaciones devolvieron resultados exactos en formato Q7.8.

Con las entradas `1.5` y `2`:

Operación	Resultado real	Resultado Q7.8
Conversión de 1.5	1.5	384 / 256 = 1.5
Suma	1.5 + 2 = 3.5	896 / 256 = 3.5
Multiplicación	1.5 × 2 = 3.0	768 / 256 = 3.0
División	1.5 / 2 = 0.75	192 / 256 = 0.75

Figura 13 - Resultados reales vs Q7.8

Asimismo se verificó que los valores enteros obtenidos coinciden bit a bit con su representación en sistema hexadecimal.

- `6144` = `x1800`
- `512` = `x0200`
- `384` = `x0180`
- `896` = `x0380`
- `768` = `x0300`
- `192` = `x00C0`

Figura 14 - Equivalencias Decimales y Hexadecimales

- Pruebas de Comportamiento Numérico: Se llevó a cabo la ejecución de las 5 pruebas restantes para evaluar signos, fracciones, el cero, el redondeo y los límites de representación.

Primer número	Segundo número	Qué comprobamos
-1.5	2	Números negativos
0.5	0.5	Resultados fraccionarios
0	5	Operaciones con cero
1.2345	2.3456	Precisión y redondeo
7.9	120	Desbordamiento en multiplicación

Figura 15 - Pruebas Complementarias

Al correr estos escenarios en el programa, se obtuvo coherencia matemática entre los valores teóricos esperados y los resultados arrojados por el código en punto fijo, incluyendo la detección del desbordamiento en el último caso.

Prueba	Operación	Resultado esperado	Resultado obtenido
1	Suma	0.5	0.5
1	Multiplicación	-3.0	-3.0
1	División	-0.75	-0.75
2	Multiplicación	0.25	0.25
2	División	1.0	1.0
3	Multiplicación	0	0
3	División	0	0
4	Multiplicación	≈ 2.8945	2.8945
4	División	≈ 0.5263	0.5273
5	Multiplicación	948	Desbordamiento detectado
5	División	≈ 0.06583	0.06641

Figura 16 - Resultado Esperado y Obtenido

A continuación se detalla el análisis individual de cada una de las pruebas ejecutadas:

- Números Negativos: Se comprobó que el redondeo manual y las rutinas aritméticas respetan correctamente el signo en complemento a 2. Tanto la suma, multiplicación como la división dieron resultados exactos.

$$-1.5 \times 2 = -3$$

El resultado almacenado fue `-768`, que en Q7.8 representa:

$$\frac{-768}{256} = -3$$

Figura 17 - Prueba con números negativos

- Resultados Fraccionarios: Se verificó que el programa no trunca resultados menores a la unidad, obteniendo con exactitud 0.25 en el producto (64/256), 1.0 en el cociente y 1.0 en la suma.
- Operaciones con 0 en el numerador: Se corroboró la correcta propagación del valor nulo tanto en la conversión entre formatos como en el producto y la división.
- Error de Cuantización y redondeo: Se demuestra el efecto de contar con una cantidad finita de bits fraccionarios. Los números ingresados no tienen representación binaria exacta en 12 bits ni en 8 bits fraccionarios, por lo que se almacenan como aproximaciones cercanas.

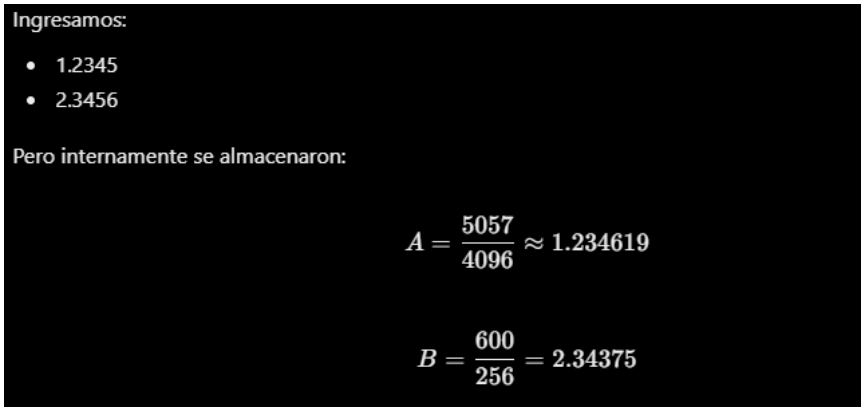


Figura 18 - Precisión y redondeo

- Desbordamiento: Se validó el uso de registros temporales de 32 bits.

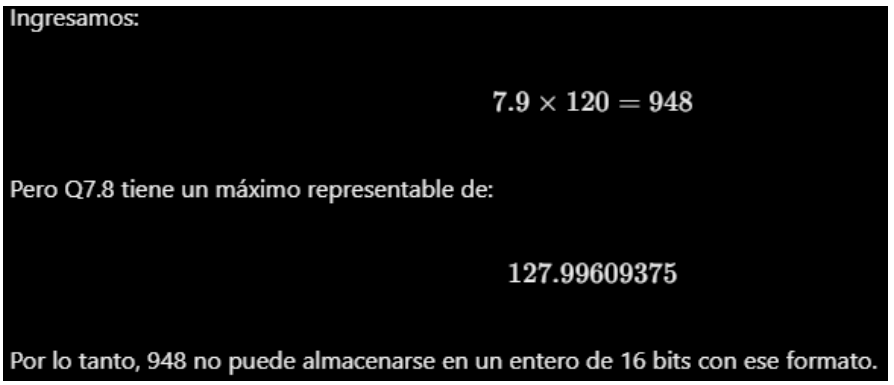


Figura 19 - Desbordamiento en multiplicación

Por último, tras la incorporación de la función validar_entrada(), se ejecutaron 3 pruebas adicionales destinadas a comprobar que el programa rechace valores fuera de los límites de los formatos Q3.12 y Q7.8 antes de realizar el casting, y que ataje la división por cero.

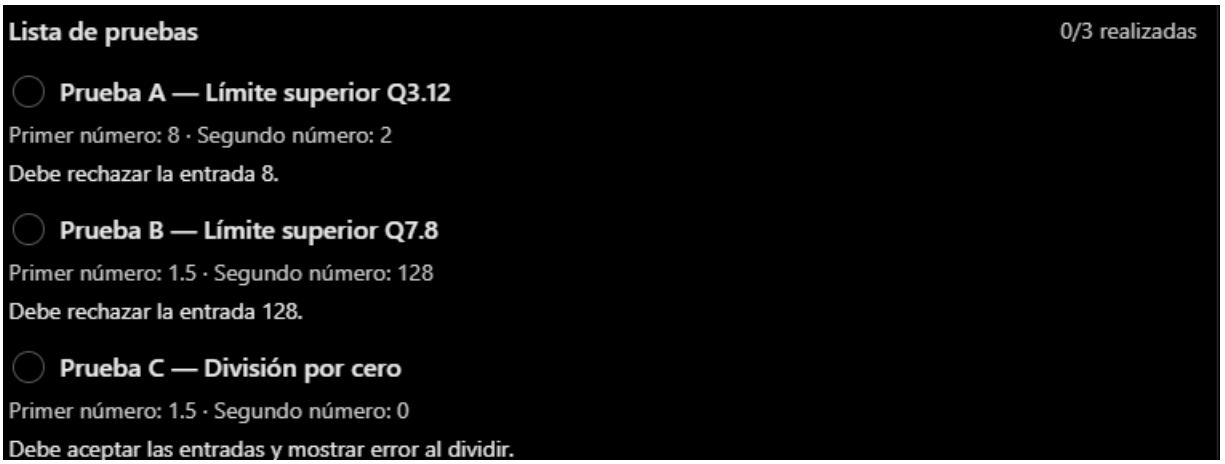


Figura 20 - Lista de Pruebas planteadas

Los resultados confirmaron el bloqueo correcto de las entradas fuera de escala (8 para Q3.12 y 128 para Q7.8) y la detección del divisor nulo sin interrumpir el cálculo previo de la suma y la multiplicación.

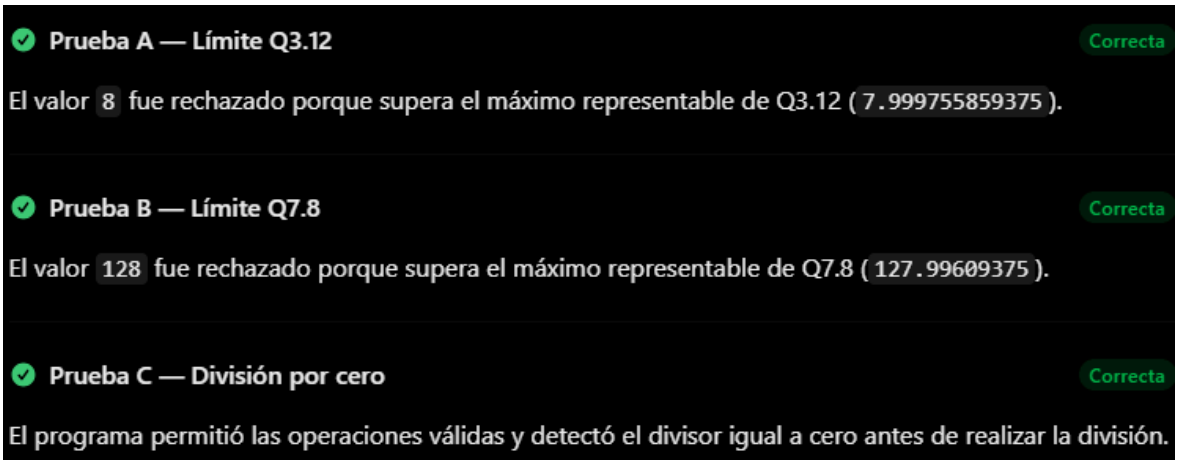


Figura 21 - Resultados obtenidos en las validaciones

4 Implementación en ASSEMBLY

En la etapa final del laboratorio se trasladaron los puntos 3 (conversión de formato Q3.12 a Q7.8) y 4 (multiplicación en punto fijo con ajuste de escala) al lenguaje ensamblador de la arquitectura LC-3. Tal como establece la consigna, los números a operar se incluyeron directamente en la memoria del procesador mediante directivas .FILL, previamente formateados como enteros con signo de 16 bits en sus respectivas escalas.

El pasaje de un lenguaje de alto nivel (C++) a la ISA de la LC-3 impuso restricciones arquitectónicas severas que obligan a rediseñar los algoritmos matemáticos a nivel de registros:

- **Ausencia de instrucciones aritméticas complejas y de desplazamiento a derecha:** La LC-3 solo cuenta con suma (ADD), negación bit a bit (NOT) y conjunción lógica (AND) como operaciones de ALU. No posee instrucciones de multiplicación, división ni desplazamientos a la derecha.
- **Simulación de desplazamientos e inspección de bits:** Todo desplazamiento a la izquierda de 1 bit se implementó sumando un registro consigo mismo (ADD Rx, Rx, Rx), mientras que la lectura del bit más significativo (bit 15) se resolvió aprovechando los códigos de condición del procesador (N, Z, P), dado que un número en complemento a 2 es negativo (BRn) si y sólo si su bit 15 es 1.
- **Alcances y limitaciones de la IA en Assembly LC-3:** Si bien la IA resultó útil para estructurar el algoritmo base, presentó fallas críticas propias de operar sobre una ISA restringida: asumió la existencia de un *Carry flag* inexistente al evaluar sumas con BRzp, omitió el caso borde asimétrico de -32768 (x8000) y las cargas iniciales (LD), y calculó erróneamente la cota de *overflow* previa al escalado (#-16 en vez de 2048). Estos defectos se identificaron y corrigieron depurando paso a paso con *breakpoints* en el simulador.

```
; Para que quepa en 16 bits, R2 debe ser menor que 16.
ADD R4, R2, #-16
BRzp OVERFLOW
```

Figura 22 - Error de lógica

4.1.1 Implementación: Rutinas en LC-3 y Validaciones

A continuación se describen las dos rutinas implementadas en lenguaje ensamblador junto con sus pruebas de verificación mediante ***breakpoints*** y observación de registros.

Punto 3 — Conversión de Q3.12 a Q7.8 con redondeo: Para alinear el punto binario reduciendo de 12 a 8 los bits fraccionarios se debe dividir el operando por $2^4 = 16$. El programa separa inicialmente el signo del número (almacenando una bandera en R5 y convirtiendo el operando a magnitud positiva en R0) y contempla mediante una rama explícita el caso límite de -32768 (x8000), al cual le asigna directamente -2048.

```
.ORIG x3000

; ========================================
; PUNTO 3: CONVERSION Q3.12 -> Q7.8
; Divide por 16 (>> 4) con redondeo
; ========================================

        LD R0, NUM1_Q3_12       ; Cargar operando en Q3.12
        AND R2, R2, #0          ; R2 = Acumulador de cociente
        AND R5, R5, #0          ; R5 = Bandera de signo (0 pos, 1 neg)

        ; Verificar si es negativo
        ADD R0, R0, #0
        BRzp ES_POSITIVO

        ; Comprobar caso borde -32768 (x8000)
        ADD R1, R0, R0
        BRz CASO_MINIMO

        ADD R5, R5, #1          ; Marcar signo negativo
        NOT R0, R0
        ADD R0, R0, #1          ; R0 = |NUM1|

ES_POSITIVO
        ; Extraer 12 bits superiores (equivale a R0 >> 4)
        AND R3, R3, #0
        ADD R3, R3, #12         ; Contador = 12 desplazamientos

BUCLE_CONV
        ADD R2, R2, R2          ; Desplazar cociente a izquierda
        ADD R0, R0, #0
        BRzp BIT_CERO_CONV
        ADD R2, R2, #1          ; Pasar bit 15 de R0 al bit 0 de R2

BIT_CERO_CONV
        ADD R0, R0, R0          ; Desplazar R0 a izquierda
        ADD R3, R3, #-1
        BRp BUCLE_CONV

        ; Redondeo: el bit 3 original ahora esta en el bit 15 de R0
        ADD R0, R0, #0
        BRzp APLICAR_SIGNO_CONV
        ADD R2, R2, #1          ; Resto >= 8 -> sumar 1

APLICAR_SIGNO_CONV
        ADD R5, R5, #0
        BRz FIN_CONV
        NOT R2, R2
        ADD R2, R2, #1          ; Restaurar signo negativo
        BRnzp FIN_CONV

CASO_MINIMO
        LD R2, VALOR_MIN_Q7_8   ; -32768 >> 4 = -2048 (xF800)

FIN_CONV
        ST R2, RES_Q7_8
        HALT

NUM1_Q3_12      .FILL #6144     ; 1.5 en Q3.12 (x1800)
VALOR_MIN_Q7_8  .FILL xF800     ; -8.0 en Q7.8 (-2048)
RES_Q7_8        .FILL #0        ; Resultado esperado: 384 (x0180)

        .END
```

Figura 23 - Conversión a Q7.8

Punto 4 (Multiplicación Q3.12 × Q7.8): Opera con magnitudes positivas y almacena el producto intermedio de 32 bits en R2:R3 mediante un bucle de 16 iteraciones. El ajuste de escala a Q7.8 se realiza mediante 4 desplazamientos adicionales a izquierda sobre R2:R3 (división por 4096), aplicando redondeo con el bit 11 original y validando el rango de 16 bits antes de aplicar el complemento a 2 definitivo según el signo registrado en R6.

```
.ORIG x3000

; ========================================
; PUNTO 4: MULTIPLICACION Q3.12 * Q7.8 -> Q7.8
; Producto 32 bits (R2:R3), escala /4096 y redondeo
; ========================================

        LD R0, NUM1
        LD R1, NUM2
        AND R6, R6, #0          ; R6 = Signo resultado (0 pos, 1 neg)

        ADD R0, R0, #0
        BRzp A_POSITIVO
        ADD R6, R6, #1
        NOT R0, R0
        ADD R0, R0, #1          ; R0 = |NUM1|

A_POSITIVO
        ADD R1, R1, #0
        BRzp B_POSITIVO
        ADD R6, R6, #0
        BRp SIGNO_POS
        ADD R6, R6, #1
        BRnzp B_MAGNITUD
SIGNO_POS
        AND R6, R6, #0
B_MAGNITUD
        NOT R1, R1
        ADD R1, R1, #1          ; R1 = |NUM2|

B_POSITIVO
        AND R2, R2, #0          ; R2 = Parte alta (32 bits)
        AND R3, R3, #0          ; R3 = Parte baja (32 bits)
        AND R5, R5, #0
        ADD R5, R5, #15
        ADD R5, R5, #1          ; R5 = 16 iteraciones

MULTIPLICAR
        ADD R3, R3, #0          ; Desplazar R2:R3 a izquierda
        BRn BIT_BAJO_UNO
        ADD R2, R2, R2
        BRnzp DESPLAZAR_BAJO
BIT_BAJO_UNO
        ADD R2, R2, R2
        ADD R2, R2, #1
DESPLAZAR_BAJO
        ADD R3, R3, R3

        ADD R1, R1, #0          ; Inspeccionar bit 15 de R1
        BRzp BIT_CERO
        ADD R3, R3, #0          ; Sumar R0 a R3 con deteccion de carry
        BRn R3_NEGATIVO
        ADD R3, R3, R0
        BRnzp FIN_SUMA
R3_NEGATIVO
        ADD R3, R3, R0
        BRn FIN_SUMA
        ADD R2, R2, #1          ; Propagar acarreo a R2
FIN_SUMA
BIT_CERO
        ADD R1, R1, R1
        ADD R5, R5, #-1
        BRp MULTIPLICAR
```

```
        ; Ajuste de escala (/4096): 4 desplazamientos extra a izquierda
        AND R5, R5, #0
        ADD R5, R5, #4
ESCALAR
        ADD R2, R2, #0
        BRn OVERFLOW            ; Si bit 15 de R2 es 1, desborda 16 bits
        ADD R3, R3, #0
        BRn ESC_BIT_UNO
        ADD R2, R2, R2
        BRnzp ESC_BAJO
ESC_BIT_UNO
        ADD R2, R2, R2
        ADD R2, R2, #1
ESC_BAJO
        ADD R3, R3, R3
        ADD R5, R5, #-1
        BRp ESCALAR

        ; Redondeo con el bit 11 original de R3 (ahora en bit 15)
        ADD R3, R3, #0
        BRzp SIN_REDONDEO
        ADD R2, R2, #1
        BRz OVERFLOW
SIN_REDONDEO

        ; Validar rango de 16 bits segun signo
        ADD R6, R6, #0
        BRp VALIDAR_NEGATIVO
        ADD R2, R2, #0
        BRn OVERFLOW
        BRnzp APLICAR_SIGNO
VALIDAR_NEGATIVO
        ADD R2, R2, #0
        BRzp APLICAR_SIGNO
        ADD R4, R2, R2          ; Admitir unicamente x8000 (-32768)
        BRnp OVERFLOW

APLICAR_SIGNO
        ADD R6, R6, #0
        BRz GUARDAR_OK
        NOT R2, R2
        ADD R2, R2, #1
GUARDAR_OK
        ST R2, RESULTADO
        AND R4, R4, #0
        ST R4, ESTADO
        HALT

OVERFLOW
        AND R2, R2, #0
        ST R2, RESULTADO
        AND R4, R4, #0
        ADD R4, R4, #1
        ST R4, ESTADO
        HALT

NUM1        .FILL #-6144
NUM2        .FILL #512
RESULTADO   .FILL #0
ESTADO      .FILL #0

        .END
```

Figura 24 - Multiplicación con ajuste de escala

Pruebas en Simulador (LC3Tutor): Con el breakpoint en x304A, se validó el caso positivo (NUM1 = **#6144**, NUM2 = **#512**), obteniendo R2 = x0300 (3. 0 en Q7.8) con ESTADO = 0. Asimismo, se verificó el caso negativo (NUM1 = **#-6144**, NUM2 = **#512**), confirmando el resultado R2 = xFD00 (− 3. 0) y la correcta persistencia de los datos en memoria (x3051 a x3054).

5 Segunda Parte: Cálculo de Seno y Coseno por Tabla

En esta segunda etapa del laboratorio se desarrolló el módulo encargado de calcular las funciones trigonométricas seno y coseno para una extensión de la arquitectura LC-3 (basada en la tercera edición de Patt y Patel) con código de operación (*opcode*) de 3 bits. El diseño debía implementarse inicialmente en C++ y luego traducirse a Assembly para ser integrado por el grupo coordinador mediante directivas .EXTERN. La premisa técnica asignada al Grupo 22 exigió utilizar **una única tabla de búsqueda (*Lookup Table*)** para ambas funciones, resolviendo la función restante mediante un desplazamiento de fase de $\pi/2$:

$$\cos(x) = \sin\left(x + \frac{\pi}{2}\right)$$

5.1.1 Planteo Metodológico:

Para abordar el cálculo trigonométrico mediante tablas de consulta (*Lookup Tables* - LUT), se planteó como premisa metodológica inicial la generación de una fuente única de verdad en formato .csv automatizada mediante un script en Python. Dicha base estructuró el índice, el ángulo en radianes, el valor matemático real, su cuantización en punto fijo y la palabra hexadecimal de 16 bits en complemento a 2, garantizando coherencia absoluta entre el prototipo en C++ y las directivas .FILL en Assembly.

Al someter el dimensionamiento de la tabla al análisis de la IA Generativa, el modelo propuso inicialmente resoluciones estándar de 256 a 1024 entradas para cubrir un período completo (360°). Sin embargo, este planteo preliminar debió reformularse al confrontarlo con el entorno real del proyecto:

- **Memoria de datos compartida:** El desarrollo no se ejecuta de forma aislada, sino como parte de una ISA extendida donde entre 10 y 15 grupos comparten el mapa de memoria de la LC-3. Alojar tablas de 512 o 2048 bytes restaba espacio crítico a los módulos de los demás grupos.
- **Prototipo V1 (8 muestras):** Como primera aproximación para contrastar con la cátedra, se diseñó un prototipo experimental acotado a solo 8 palabras en memoria. Se evaluó la posibilidad de compensar la baja resolución angular reutilizando subrutinas aritméticas de otros grupos (como multiplicación o división en punto fijo) mediante directivas .EXTERN.
- **Ajuste por directiva docente (Hacia el Prototipo V2):** En la consulta con el profesor de cátedra se descartó la dependencia de operaciones externas complejas, indicando que el módulo debía ser autónomo y resolver las aproximaciones intermedias mediante un promedio entre muestras consecutivas. Asimismo, se estableció que la cantidad de entradas debía comenzar en un piso provisional de 100 valores (200 bytes), dejando la conversión de la interfaz de entrada bajo la responsabilidad del grupo coordinador.

5.2 Decisiones Técnicas y Ajustes de Arquitectura:

A partir de las restricciones de hardware y la devolución docente, se consolidaron tres decisiones de diseño:

- **Formato Q1.14 y desacople de entrada**: Se adoptó Q1.14 (escala $2^{14} = 16384$) para las constantes y salidas, mapeando $[-1, 1]$ con máxima resolución (1.0 = x4000). Al advertir que su límite superior (≈ 1.9999) imposibilita representar ángulos mayores a $2rad$ (como π o 2π), se reservó Q1.14 exclusivamente para los valores tabulados y el resultado final, desacoplándolo del formato del ángulo de entrada.

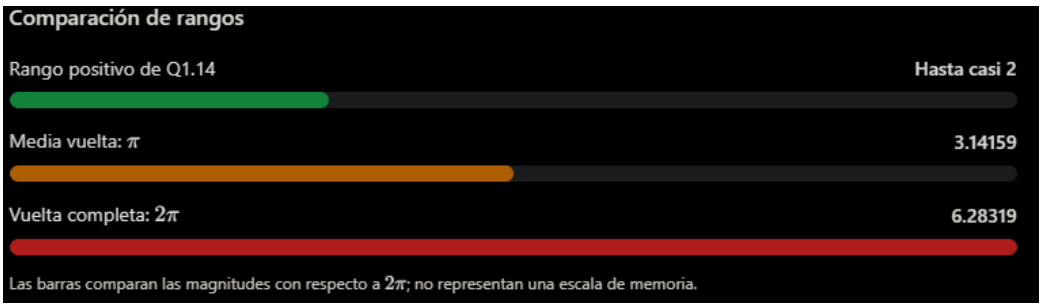


Figura 25 - Tabla comparativa de Rangos

- **Muestreo en cuarto de onda ($[0, \pi/2]$)**: Distribuir las 100 muestras de 0 a π genera redundancia por la simetría $\sin(\pi - \theta) = \sin(\theta)$. Confinar la tabla al primer cuadrante reduce el paso angular a la mitad ($h \approx 0.01587rad$) y divide por cuatro la cota teórica de error de interpolación ($E \approx 0.0000315$), a cambio de apenas 4 a 10 instrucciones en LC-3 para resolver cuadrantes y signos.
- **Cálculo intermedio (Promedio vs. Interpolación)**: Ante la pauta de promediar valores, se diferenció el promedio simple $\frac{y_0+y_1}{2}$ (exacto únicamente en el punto medio) de la interpolación lineal ponderada, necesaria para garantizar precisión frente a ángulos ubicados en cualquier posición del intervalo.

5. 3 Diseño del Algoritmo y Optimización del Dominio:

Con las especificaciones acordadas, se estructuró el algoritmo base en torno a tres componentes esenciales: la reducción angular por simetrías, la indexación sobre la tabla de consulta y la estimación intermedia.

- **Optimización del Dominio (Cuarto de onda frente a Media onda)**: Para distribuir las 100 muestras en memoria, se evaluó si convenía muestrear medio ciclo ($[0, \pi]$) o restringir la tabla al primer cuadrante ($[0, \pi/2]$). Confinar las 100 muestras a $[0, \pi/2]$ elimina los datos redundantes generados por la simetría $\sin(\pi - \theta) = \sin(\theta)$, reduciendo el paso angular a la mitad ($h \approx 0.01587rad$ frente a $0.03173rad$). En consecuencia, la cota teórica de error de interpolación:

$$E \leq \frac{h^2}{8}$$

desciende a ≈ 0.0000315, cuatro veces menor que en media onda (≈ 0.000126), a cambio de apenas 4 a 10 instrucciones adicionales en LC-3 para resolver cuadrantes.

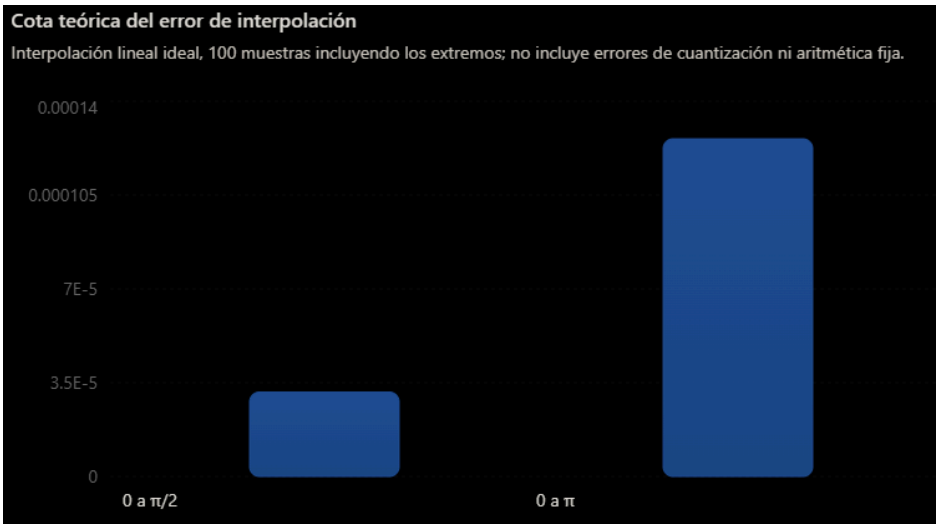


Figura 26 - Error de interpolación

- **Mapeo angular y cálculo unificado de Seno y Coseno**: Cualquier ángulo $\theta \in [0, 2\pi]$ se traslada al primer cuadrante mediante identidades de simetría:
  - Cuadrante I ($[0, \pi/2]$): $\theta' = \theta$, signo positivo.
  - Cuadrante II ($[\pi/2, \pi]$): $\theta' = \pi - \theta$, signo positivo.
  - Cuadrante III($[\pi, 3\pi/2]$): $\theta' = \theta - \pi$, signo negativo.
  - Cuadrante IV ($[3\pi/2, 2\pi]$): $\theta' = 2\pi - \theta$, signo negativo.

Para obtener el coseno a partir de la misma y única tabla de seno, se aplica el desfasaje angular fundamental:

$$\cos(\theta) = \sin\left(\theta + \frac{\pi}{2}\right)$$

o bien la relación de ángulo complementario $\cos(\theta) = \sin(\pi/2 - \theta)$, resolviendo ambas funciones con una única estructura de datos en memoria.

5.3.1 Dilema Algorítmico: Series de Taylor frente a Tablas de Consulta

Durante el proceso asistido por IA, al solicitarle al modelo que evitara las funciones de biblioteca de <cmath> para no operar con "cajas negras", la IA propuso de forma automática calcular el seno y el coseno en tiempo de ejecución implementando aproximaciones polinómicas mediante Series de Taylor:

$$\sin(x) \approx x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots = x - \frac{x^3}{6} + \frac{x^5}{120}$$

$$\cos(x) \approx 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots = 1 - \frac{x^2}{2} + \frac{x^4}{24}$$

Figura 27 - Series de Taylor

Verificación preliminar de las series de Taylor:
Para comprobar la precisión de las aproximaciones polinómicas propuestas, se evaluaron manualmente los primeros términos para $x = 0\,rad$ y $x = 1.0\,rad$:
Evaluación en $x = 0\,rad$:

$$\sin(0) \approx 0 - \frac{0^3}{6} + \frac{0^5}{120} = 0$$

$$\cos(0) \approx 1 - \frac{0^2}{2} + \frac{0^4}{24} = 1$$

(Coincidencia exacta con los valores de referencia).

Evaluación en $x = 1.0\,rad$:

$$\sin(1) \approx 1 - \frac{1}{6} + \frac{1}{120} \approx 0.841666\ (real: 0.841471; E_{abs} \approx 0.000195)$$

$$\cos(1) \approx 1 - \frac{1}{2} + \frac{1}{24} \approx 0.541667\ (real: 0.540302; E_{abs} \approx 0.001365)$$

Los errores analíticos obtenidos confirmaron una aproximación adecuada con apenas tres términos.

Inviabilidad de ejecución en tiempo real en la LC-3:
La implementación directa del polinomio en ensamblador se descartó por dos limitaciones de hardware:

- **Restricciones de la ALU**: Al disponer únicamente de ADD, AND y NOT, calcular potencias y factoriales exige multiplicaciones y divisiones por software mediante bucles iterativos, incrementando de forma inviable el conteo de ciclos de instrucción.

- **Desbordamiento en punto fijo**: Al operar con el ángulo en formato $Q3.12$ ($x_{entero} = x \cdot 4096$), el cálculo de x^2 produce un valor intermedio en $Q6.24$:

$$x^2_{intermedio} = 4096 \times 4096 = 16.777.216$$

Este resultado excede el ancho nativo de 16 bits, requiriendo acumulación en 32 bits y rutinas adicionales de desplazamiento para reescalar a $Q1.14$.

Resolución: Arquitectura desacoplada en dos etapas
Para suprimir el costo computacional en tiempo de ejecución, el sistema se estructuró en dos dominios:

- **Etapa estática (C++)**: Ejecuta el cálculo analítico fuera del procesador para generar las 100 constantes de referencia del primer cuadrante y exportarlas como directivas .FILL.
- **Etapa dinámica (LC-3 Assembly)**: Omite el cálculo de series en tiempo real; se limita a indexar la tabla en memoria, resolver cuadrantes y signos mediante simetría y aproximar los puntos intermedios por promedio aritmético, utilizando únicamente instrucciones nativas de 16 bits.

5.3.2 Cálculo Intermedio (Promedio vs. Interpolación)

Para estimar valores angulares que caen entre dos muestras consecutivas de la tabla ($y_0 = \sin(\theta_0)$ e $y_1 = \sin(\theta_1)$), se plantearon dos aproximaciones matemáticas:

- **Promedio Aritmético Simple:**

$$y \approx \frac{y_0+y_1}{2}$$

  Constituye la instrucción sugerida por la cátedra para mantener la independencia del módulo. Su ventaja crítica en **LC-3** es que no requiere multiplicaciones: se implementa sumando ambos registros de 16 bits (ADD R2, R0, R1) y ejecutando un desplazamiento aritmético hacia la derecha de 1 bit. Es matemáticamente exacto únicamente si el ángulo se ubica en el punto medio del intervalo ($\theta = \theta_0 + h/2$).

- **Interpolación Lineal Ponderada:**

$$y \approx y_0 + \frac{\theta-\theta_0}{h}(y_1 - y_0)$$

  Garantiza uniformidad del error en todo el intervalo continuo, pero exige calcular el producto de la distancia fraccionaria por la pendiente local, demandando operaciones de multiplicación en punto fijo.

6 Implementación y Pruebas Unitarias en C++

Para asegurar la consistencia numérica de las constantes antes de construir la tabla, se desarrolló y validó de forma aislada la función de conversión a punto fijo.

- **Tipos de datos y redondeo simétrico**:

  Se utilizaron tipos int16_t para emular el ancho de palabra de la LC-3 y float para las variables auxiliares. Se descartó round() de <cmath> en favor de un redondeo explícito ($\pm\ 0.5f$) según el signo del valor escalado ($2^{14} = 16384$):

```
int16_t convertirAQ14(float valor) {
    float escalado = valor * 16384.0f;

    if (escalado >= 0.0f)
        return (int16_t)(escalado + 0.5f);
    else
        return (int16_t)(escalado - 0.5f);
}
```

Figura 28 - Función Convertir

- **Validación experimental**:

  La función se evaluó bajo tres escenarios:

  - Casos base de control: Signos positivos ($0.5 \rightarrow 8192$), negativos ($-\ 0.5 \rightarrow -\ 8192$) y cero ($0.0 \rightarrow 0$).
  - Valores fraccionarios: Comprobación de truncamiento con 0.1 ($1638.4 \rightarrow 1638$) y $-\ 0.1$ ($-\ 1638.4 \rightarrow -\ 1638$).
  - Umbrales de desempate ($\pm\ 0.5$): Evaluación en la frontera crítica de redondeo ($10.5 \rightarrow 11$, $-\ 10.5 \rightarrow -\ 11$, $10.49 \rightarrow 10$ y $-\ 10.49 \rightarrow -\ 10$).

Todas las pruebas arrojaron coincidencia plena con los valores analíticos, validando la función **convertirAQ14()** como la base numérica confiable para estructurar el generador de la tabla de muestras.

6.1 El Polinomio y el Giro de 100 a 91 Muestras (1° por Entrada)

Tras validar **convertirAQ14()**, el equipo se enfrentó al problema de cómo generar los valores de la tabla sin depender de funciones prefabricadas. Si bien inicialmente se había fijado un piso arbitrario de 100 muestras para el intervalo $[0, \pi/2]$, dividir 90° en 100 partes producía un paso fraccionario incómodo (0.9° o $\approx 0.01587 rad$ por índice), lo que complejizaba innecesariamente la lógica posterior para mapear un ángulo a una posición de memoria. Durante las pruebas, surgió desde el equipo la propuesta de reformular el tamaño de la tabla de 100 a 90 valores, asignando exactamente 1 muestra por cada grado sexagesimal.

Al analizar los bordes del intervalo con la IA, se detectó un detalle crítico de indexación: para cubrir desde 0° hasta 90° inclusive a razón de un grado por posición, no bastaban 90 entradas (que solo cubrirían de 0° a 89°), sino que se requerían 91 muestras (índices 0 a 90), ocupando 182 bytes de memoria.

Con este nuevo dimensionamiento de 91 puntos, se ejecutó un segundo banco de pruebas en C++ comparando las funciones manuales basadas en Series de Taylor (7 términos) contra sin() y cos() de <cmath> :

- **Error en decimales (punto flotante):**

  La diferencia máxima fue prácticamente nula:

  - En el seno: apenas **0.00000012**(registrado en el ángulo de 66°).
  - En el coseno: apenas **0.00000006**(registrado en el ángulo de 8°).
- **Diferencia real al pasarlo a enteros Q1.14: 0 unidades**

  En formato Q1.14, el escalón más chico que puede registrar la memoria es $1/16384 \approx 0.000061$. Como el error del polinomio (0.00000012) es más de 500 veces más chico que ese escalón, el redondeo lo absorbe por completo. En los 91 grados probados, el entero obtenido fue exactamente el mismo que si hubiéramos usado la calculadora científica ideal.

```
// Verificación del error grado por grado frente a <cmath>
for (int grado = 0; grado <= 90; grado++) {
    float angulo = grado * (3.14159265f / 180.0f);

    // Comparación del valor entero cuantizado a Q1.14
    int difSeno = convertirAQ14(calcularSeno(angulo)) - convertirAQ14(sin(angulo));

    if (abs(difSeno) > errorMaxQ14Seno)
        errorMaxQ14Seno = abs(difSeno);
// Arrojó 0 unidades en las 91 muestras
}
```

Figura 29 - Verificación grado por grado

6.2 ¿Tabla Bidimensional (X, Y) o Vector Único?

Al pasar del cálculo matemático a la estructuración de la tabla en código, surgió una duda fundamental de diseño en el grupo: ¿cómo debía buscar la LC-3 el ángulo dentro de la tabla? La intuición inicial fue plantear un esquema de dos columnas o vectores paralelos: una columna X que almacenará los ángulos en radianes y una columna Y con el resultado de cada seno, buscando coincidencia en X para devolver Y.

Sin embargo, este enfoque duplica el consumo de memoria (requiriendo 182 palabras en lugar de 91) y obligaba a realizar búsquedas comparativas lentas en Assembly. Al discutir el problema, se destrabó la simplificación clave del módulo: dado que las muestras están espaciadas uniformemente cada 1° desde 0°, el ángulo X queda representado implícitamente por el propio índice de memoria (Posición 0 = 0°, Posición 1 = 1°,Posición 90 = 90°).

De este modo, se descartó almacenar los ángulos y se conservó únicamente un vector unidimensional con los resultados en Q1.14.

6.3 Errores en la Generación de .FILL y el Caso Borde de 16384

Para volcar las 91 muestras al ensamblador, se cargó primero un arreglo en C++ con los senos expresados con 15 decimales en tipo float y se automatizó la impresión por consola de las directivas .FILL #valor.

En esta etapa se debieron corregir dos obstáculos técnicos:

- **Pérdida silenciosa de precisión y error de declaración de tipos**: Aunque se escribieron constantes con 15 decimales en el código fuente, el tipo float solo retiene entre 6 y 7 cifras significativas (suficientes igualmente para cubrir la escala de 16384). Además, en la primera versión del script generador se declaró por error el vector de salida como flotante (float Q1_14[91]) en lugar de entero (int Q1_14[91]), lo cual fue corregido antes de exportar las líneas .FILL para asegurar que el ensamblador recibiera enteros puros.
- **Duda de desbordamiento en el extremo** $\sin(90^{\circ}) = 1.0$: En las revisiones surgió la alerta de si el valor máximo de la tabla ($1.0 \times 16384 = 16384$) desbordaría un entero con signo de 16 bits. Se verificó que en complemento a 2 el límite positivo es 32767 (x7FFF); por lo tanto, 16384 (x4000) entra perfectamente como número positivo (representando el 1.0 exacto en Q1.14, cuyo tope es ≈ 1.99994).

Una vez depurado el generador, las 91 directivas .FILL se probaron directamente en el ensamblador online desarrollado por otro grupo de la cátedra, logrando ensamblar la tabla en memoria sin errores de sintaxis.

6.4 Lógica de Consulta: Formatos, Cuadrantes e Índices

Con la tabla cargada, el mayor desafío consistió en traducir el ángulo de entrada hacia las posiciones 0 a 90 de memoria utilizando únicamente las instrucciones nativas de la LC-3 (ADD, AND, NOT, cargas y bifurcaciones).

Errores Conceptuales y Ajustes de Diseño

Durante el planteo inicial del algoritmo surgieron tres confusiones que obligaron a redefinir el flujo de control:

- **Ángulo de entrada frente a índice de tabla:** Se aclaró que el dato no puede usarse directamente como dirección de memoria. Al recibirse en radianes y en punto fijo $Q3.12$ (escalado por 4096), un valor como 2 representa $2rad$ ($\approx 114.59^{\circ}$), por lo que exige un reescalado previo al rango de índices de la tabla (0 a 90)
- **Cálculo del coseno por ángulo complementario:** Se corrigió el intento inicial de restar 90° al resultado del seno (90° - resultado_seno). La identidad$\cos(\theta) = \sin(90^{\circ} - \theta)$ opera sobre la posición de memoria, no sobre la amplitud: para 2°, el seno consulta TABLA_SENO[2] y el coseno TABLA_SENO[90 - 2] = TABLA_SENO[88]..
- **Determinación previa del cuadrante:** Se descartó deducir el cuadrante evaluando los signos del seno y coseno mediante saltos condicionales (BR), ya que resultaba una lógica circular: la tabla solo almacena magnitudes positivas del primer cuadrante. El flujo correcto evalúa primero la magnitud angular contra las fronteras (π/2,π,3π/2), reduce el argumento al intervalo [0,90°], lee la tabla y recién al final aplica el signo correspondiente al cuadrante identificado.

Radianes (Q3.12) vs. Grados (Q7.8) y el Costo de Conversión

Al notar que recibir radianes obligaba a convertir el ángulo a grados (e incluir lógica de redondeo en Assembly) para poder indexar las 91 posiciones de la tabla, el equipo puso en duda si convenía mantener la entrada en radianes bajo formato **Q3.12** o cambiar directamente la entrada a grados sexagesimales usando **Q7.8**.

Se analizaron las consecuencias de ambas posturas:

- **Por qué fracasó la alternativa de Grados en Q7.8:** Aunque recibir grados parecía ahorrar la conversión a índices de 1°, el formato Q7.8 con signo (1 bit de signo, 7 bits enteros y 8 fraccionarios) solo puede representar valores hasta $+ 127.996^{\circ}$. Esto hacía imposible representar ángulos del segundo, tercer y cuarto cuadrante (hasta 360°) sin desbordar la palabra de 16 bits.
- **Decisión final:** Se ratificó mantener la entrada en **radianes en formato Q3.12** (rango $[-8, 7.9998rad]$). Asimismo, se documentó que al mapear una entrada de alta resolución (Q3.12) hacia una tabla discreta de 91 valores sin interpolación, se asume una pérdida de resolución de hasta $\pm 0.5^{\circ}$ al redondear al índice entero más cercano, priorizando la simplicidad de instrucciones en la LC-3.

6.5 Clasificador de Cuadrantes:

Superado el debate sobre los formatos y con la tabla de 91 muestras ya ensamblada en el simulador online del proyecto, el desarrollo avanzó hacia la construcción del motor de decisión angular. Esta etapa consistió en diseñar el mecanismo de saltos condicionales para la LC-3 y depurar mediante un prototipo espejo en C++ que imita las limitaciones físicas de los registros del procesador.

Sustitución de CMP y SUB mediante Aritmética en Complemento a Dos

Dado que la arquitectura LC-3 original carece tanto de una instrucción de comparación (CMP) como de una de resta (SUB), surgió en el equipo la duda sobre cómo evaluar si el ángulo era menor, igual o mayor que cada límite ($\pi/2, \pi, 3\pi/2$) sin duplicar ni complejizar la lógica en cada paso.

Se resolvió aplicar un esquema repetitivo de comparación en cascada: conservar siempre el ángulo original θ (en formato $Q3.12$) intacto en el registro R0, cargar en R2 el valor negativo precalculado de cada frontera y guardar la suma en R1 ($R_1 = R_0 + (-Límite)$). Como la instrucción ADD actualiza automáticamente los códigos de condición (N, Z, P), los saltos se ejecutan inmediatamente después de cada suma:

```
; Primera comparación contra pi/2 (6434 en Q3.12)
LD  R2, NEG_PI_2      ; R2 = -6434
ADD R1, R0, R2        ; R1 = theta - 6434 (actualiza códigos N, Z, P)
BRn CUADRANTE_1       ; Si dio negativo (theta < pi/2), pertenece al Cuadrante I
BRz CASO_PI_2         ; Si dio cero (theta == pi/2), salta al Caso Especial de 90°

; Si no saltó ninguno, el resultado es positivo (theta > pi/2).
; No hace falta un BRp explícito; continúa evaluando el mismo R0 contra pi:
LD  R2, NEG_PI        ; R2 = -12868
ADD R1, R0, R2        ; R1 = theta - 12868
BRn CUADRANTE_2       ; Si dio negativo (theta < pi), pertenece al Cuadrante II
BRz CASO_PI           ; Si dio cero (theta == pi), salta al Caso Especial de 180°

; Tercera comparación contra 3*pi/2 (19302 en Q3.12)
LD  R2, NEG_3PI_2     ; R2 = -19302
ADD R1, R0, R2        ; R1 = theta - 19302
BRn CUADRANTE_3       ; Si dio negativo (theta < 3*pi/2), pertenece al Cuadrante III
BRz CASO_3PI_2        ; Si dio cero (theta == 3*pi/2), salta al Caso Especial de 270°
BR  CUADRANTE_4       ; Si superó 3*pi/2, cae por descarte en el Cuadrante IV

NEG_PI_2   .FILL #-6434
NEG_PI     .FILL #-12868
NEG_3PI_2  .FILL #-19302
```

Figura 30 - Comparaciones

Este diseño trajo dos optimizaciones directas para el código de máquina:

1. **Preservación del operando de entrada (R0):** Cada comparación reutiliza el ángulo original guardado en R0, no el resultado de la resta anterior (R1), evitando recargas desde memoria.
2. **Eliminación de saltos redundantes (BRp):** Si no se cumple ni BRn ni BRz, el resultado es forzosamente positivo y el flujo cae directamente en la siguiente comparación sin gastar una instrucción de salto adicional.

6.6 "Casos Especiales" sobre los Ejes Cartesianos

Al estructurar las comparaciones contra los límites angulares surgió una duda en el equipo sobre qué representaban exactamente los llamados "casos especiales". Se definió que corresponden a los cinco ángulos que caen con exactitud sobre los ejes cartesianos: 0, $\pi/2$, π, $3\pi/2$ y 2π (cuyos equivalentes redondeados en $Q3.12$ son 0, 6434, 12868, 19302 y 25736).

Matemáticamente, estos puntos actúan como fronteras divisorias y no pertenecen de forma estricta a ningún cuadrante. En lugar de forzarlos dentro de las fórmulas generales —lo que obligaría a calcular un ángulo de referencia, buscar en dos posiciones de TABLA_SENO y aplicar una lógica de signos que no contempla el valor nulo— se decidió interceptarlos apenas la resta contra el límite da exactamente cero (mediante el salto BRz en la LC-3 o diferencia == 0 en el prototipo de C++).

Para modelar este comportamiento antes de escribir el código final, se estableció un esquema de tres estados para los indicadores de salida (1 = positivo, -1 = negativo y 0 = función nula sobre el eje), los cuales se traducen directamente a los valores extremos en punto fijo $Q1.14$ (representados en complemento a dos en la LC-3):

Ángulo	Grados	Entero Q3.12	Condición de Captura	Seno / Coseno	Salida Directa Seno (Q1.14)	Salida Directa Coseno (Q1.14)
0	0°	0	Evaluación inicial (== 0)	0 / +1	0	16384
$\frac{\pi}{2}$	90°	6434	$\theta - 6434 = 0$ (BRz)	+1 / 0	16384	0
π	180°	12868	$\theta - 12868 = 0$ (BRz)	0 / −1	0	−16384
$\frac{3\pi}{2}$	270°	19302	$\theta - 19302 = 0$ (BRz)	−1 / 0	−16384	0
2π	360°	25736	Evaluación inicial (== 25736)	0 / +1	0	16384

Figura 31 - Casos Especiales

Separar estos cinco bordes del flujo principal resolvió tres problemas prácticos de implementación:

- **Protección de los cruces por cero**: Cuando el seno o el coseno valen exactamente 0, la lógica de signos ($\pm$ 1) por sí sola no basta; asignarlos por separado evita aplicar inversiones de signo innecesarias sobre el cero.
- **Ahorro de accesos a memoria**: Para un ángulo como $\pi/2$, el resultado ($\sin = 1, \cos = 0$) es conocido de antemano, por lo que devolver 16384 y 0 en forma directa ahorra el cálculo del índice complementario y las lecturas de tabla.
- **Eliminación de ambigüedades de borde**: Evita que un ángulo límite quede mal clasificado entre dos cuadrantes contiguos y asegura que las entradas cercanas que redondeen a 6434, 12868, 19302 o 25736 reciban un tratamiento consistente en toda la arquitectura.

6.7 Evolución del Prototipo en C++:

Con la garantía de la cátedra de que el grupo de interfaz entregará siempre un ángulo validado dentro del rango $[0, 2\pi]$, el equipo propuso construir primero un prototipo en C++ que replicara fielmente el camino de datos de la LC-3 (usando únicamente restas y condicionales < 0 y $== 0$, equivalentes a BRn y BRz) antes de programar los registros en Assembly.

La construcción de este programa atravesó tres iteraciones de diseño impulsadas por las correcciones del equipo sobre el código sugerido por la IA:

- **Iteración 1** El problema de la entrada entera en crudo:

  La primera versión pedía ingresar por teclado directamente el entero ya escalado en $Q3.12$ (por ejemplo, obligaba a tipear 4096 para probar $1rad$ o 8192 para $2rad$):

```
// Iteración 1 (descartada): obligaba a calcular a mano el entero Q3.12 antes de probar
int angulo;
cout << "Ingrese el angulo en Q3.12: ";
cin >> angulo;
```

Figura 32 - Código ingreso de ángulo

- **Iteración 2** Entrada decimal (float) e introducción innecesaria de 32 bits (int32_t): Para facilitar pruebas reales, el equipo exigió ingresar el ángulo en formato decimal y convertirlo en el acto a Q3.12 antes de evaluar su cuadrante, descartando un paso intermedio por Q15 dado que el entero Q3.12 permite comparar directamente contra las constantes. Al implementar convertirAQ312(float valor), la IA incorporó de forma preventiva variables de 32 bits para evitar posibles desbordamientos en las restas.

```
// Iteración 2: uso de int32_t por precaución ante posibles desbordamientos
int16_t convertirAQ312(float valor) {
    float escalado = valor * 4096.0f;
    int32_t resultado = (escalado >= 0.0f) ? (int32_t)(escalado + 0.5f)
                                           : (int32_t)(escalado - 0.5f);
    return (int16_t)resultado;
}
// En main():
int32_t diferencia = (int32_t)anguloQ3_12 - PI_2;
```

Figura 33 - Código iteración

- **Iteración 3** — Estandarización a 16 bits (int16_t): Ante la consulta sobre el uso de 32 bits en una arquitectura de 16 bits como la LC-3, se verificó matemáticamente que el ángulo está acotado entre 0 y 2π (0 a 25736 en $Q3.12$). Por ello, el valor convertido y las restas contra las constantes caben sin riesgo de desbordamiento en un entero con signo de 16 bits (-32768 a 32767). Se eliminaron las variables de 32 bits, usando float solo para la entrada decimal y int16_t para el resto de la lógica.

6.8 Incorporación del Cálculo de Índice y Ángulo de Referencia

Una vez depurada la clasificación a 16 bits, se acopló al prototipo la etapa encargada de transformar el ángulo de referencia $\theta_{ref} \in [0, 6434]$ en una posición válida de TABLA_SENO (índices 0 a 90).

Dado que la tabla avanza a razón de 1° por posición, el índice se obtuvo pasando la referencia de $Q3.12$ a radianes reales (dividiendo por 4096), convirtiéndola a grados sexagesimales ($\times \frac{180}{\pi}$) y redondeando al entero más cercano:

$$grados = \frac{referencia_{Q3.12}}{4096} \cdot \frac{180}{\pi}$$

```cpp
#include <iostream>
#include <stdint.h>
using namespace std;

int16_t convertirAQ312(float valor) {
    float escalado = valor * 4096.0f;
    if (escalado >= 0.0f) return (int16_t)(escalado + 0.5f);
    else                  return (int16_t)(escalado - 0.5f);
}

int calcularIndice(int16_t referenciaQ312) {
    // Conversión de validación en C++: de referencia Q3.12 a grados redondeados (0 a 90)
    float grados = ((float)referenciaQ312 / 4096.0f) * (180.0f / 3.14159265f);
    return (int)(grados + 0.5f);
}

int main() {
    const int16_t PI_2 = 6434, PI = 12868, TRES_PI_2 = 19302, DOS_PI = 25736;
    float anguloDecimal;
    int16_t anguloQ3_12, diferencia, referencia;
    int cuadrante, signoSeno, signoCoseno;

    cout << "Ingrese el angulo en radianes: ";
    cin >> anguloDecimal;

    // 1. Convertir la entrada decimal a Q3.12 (16 bits puros)
    anguloQ3_12 = convertirAQ312(anguloDecimal);
```

```cpp
// 2. Identificar el cuadrante y calcular la referencia (espejo de saltos BR en LC-3)
if (anguloQ3_12 == 0 || anguloQ3_12 == DOS_PI) {
    referencia = 0; cuadrante = 0; signoSeno = 0; signoCoseno = 1;
} else {
    diferencia = anguloQ3_12 - PI_2;
    if (diferencia < 0) {
        cuadrante = 1; referencia = anguloQ3_12; signoSeno = 1; signoCoseno = 1;
    } else if (diferencia == 0) {
        cuadrante = 0; referencia = PI_2; signoSeno = 1; signoCoseno = 0;
    } else {
        diferencia = anguloQ3_12 - PI;
        if (diferencia < 0) {
            cuadrante = 2; referencia = PI - anguloQ3_12; signoSeno = 1; signoCoseno = -1;
        } else if (diferencia == 0) {
            cuadrante = 0; referencia = 0; signoSeno = 0; signoCoseno = -1;
        } else {
            diferencia = anguloQ3_12 - TRES_PI_2;
            if (diferencia < 0) {
                cuadrante = 3; referencia = anguloQ3_12 - PI; signoSeno = -1; signoCoseno = -1;
            } else if (diferencia == 0) {
                cuadrante = 0; referencia = PI_2; signoSeno = -1; signoCoseno = 0;
            } else {
                cuadrante = 4; referencia = DOS_PI - anguloQ3_12; signoSeno = -1; signoCoseno = 1
            }
        }
    }
}

// 3. Obtener el índice de la tabla a partir de la referencia reducida
int indice = calcularIndice(referencia);

cout << "Entero Q3.12: " << anguloQ3_12 << " | Indice de tabla: " << indice << endl;
cout << "Cuadrante: " << cuadrante << " | Referencia Q3.12: " << referencia << endl;
cout << "Signo del seno: " << signoSeno << " | Signo del coseno: " << signoCoseno << endl;
return 0;
```

Figura 34 - Cálculo de Índice

Al realizar la primera prueba de esta versión con $\theta = 2rad$, la consola devolvió:

Entero Q3.12: 8192 | Cuadrante: 2 | Referencia Q3.12: 4676 ($1.1416rad$**) | Indice de tabla: 65.**
Este resultado disparó un cuestionamiento clave en el grupo: **¿cómo podían** $2\ radianes$ **convertirse en un índice de** 65°**, si** $2\ radianes$ **superan los** 100°**?**
El análisis del flujo permitió clarificar la diferencia entre el ángulo original y el ángulo de referencia:

- El ángulo original de $2rad$ equivale efectivamente a $2 \times \frac{180^{\circ}}{\pi} \approx 114.59^{\circ}$, cayendo en el segundo cuadrante.
- Como la tabla solo almacena de 0° a 90°, el programa no convierte los $2rad$ (8192) a grados, sino que primero calcula cuánto le falta para llegar a 180° (π): en grados, $180^{\circ} - 114.59^{\circ} = 65.41^{\circ}$; en punto fijo $Q3.12$, $12868 - 8192 = 4676$ ($\approx 1.1416rad$).
- Recién sobre esa referencia aguda (4676) se aplica la conversión a grados, dando 65.41° y redondeando al índice 65 (sin 65°), mientras que la pérdida de los 0.41° restantes corresponden al error de cuantización sin interpolación.

6.9 Cuatro Cuadrantes y Cinco Casos Especiales

Aclarado el funcionamiento del índice, se ejecutó la validación exhaustiva del prototipo barriendo ángulos intermedios en los cuatro cuadrantes (incluyendo $2.4rad \rightarrow 9830$) y los cuatro casos testigo principales ($1, 2, 4\ y\ 5\ rad$):

- **Corrección de estimación en el Cuadrante IV ($5rad$)**: Antes de correr la prueba de $5rad$ (20480 en $Q3.12$), la IA había anticipado teóricamente que la referencia equivaldría a 73.83°. Al ejecutar el código, la resta $25736 - 20480 = 5256$ ($1.2832rad$) arrojó en realidad 73.52°, confirmando que el cálculo numérico del programa era el exacto y que ambos convergían al índice 74.

Entrada (rad)	Ángulo Real	Entero Q3.12	Cuadrante	Cálculo de Referencia (Q3.12)	Referencia (rad / Grados)	Índice	Signos (sin/cos)
1.0	57.30°	4096	I	4096	1.0000 rad ≈ 57.30°	57	+1 / +1
2.0	114.59°	8192	II	12868 – 8192 = 4676	1.1416 rad ≈ 65.41°	65	+1 / −1
2.4	137.51°	9830	II	12868 – 9830 = 3038	0.7417 rad ≈ 42.50°	43	+1 / −1
4.0	229.18°	16384	III	16384 – 12868 = 3516	0.8584 rad ≈ 49.20°	49	−1 / −1
5.0	286.48°	20480	IV	25736 – 20480 = 5256	1.2832 rad ≈ 73.52°	74	−1 / +1

Figura 35 - Cálculo y salida de índices

Posteriormente, se probaron en consola los cinco casos especiales sobre los ejes cartesianos (0, π/2, π, 3π/2 y 2π). Estas pruebas permitieron verificar dos condiciones críticas de contorno: que las salidas nulas (0) queden blindadas sin contaminarse con signos, y que en π/2 (1.5708) y 3π/2 (4.7124) la referencia 6434 se mapee exactamente al índice 90 (último elemento de la tabla de 91 valores) sin desbordar la memoria:

```
Ingrese el angulo en radianes: 0
Entero Q3.12: 0     | Caso especial: 0 o 2*pi | Indice de tabla: 0
Cuadrante: 0        | Referencia Q3.12: 0     | Signo seno: 0  | Signo coseno: 1

Ingrese el angulo en radianes: 1.5708
Entero Q3.12: 6434  | Caso especial: pi/2     | Indice de tabla: 90
Cuadrante: 0        | Referencia Q3.12: 6434  | Signo seno: 1  | Signo coseno: 0

Ingrese el angulo en radianes: 3.1416
Entero Q3.12: 12868 | Caso especial: pi       | Indice de tabla: 0
Cuadrante: 0        | Referencia Q3.12: 0     | Signo seno: 0  | Signo coseno: -1

Ingrese el angulo en radianes: 4.7124
Entero Q3.12: 19302 | Caso especial: 3*pi/2   | Indice de tabla: 90
Cuadrante: 0        | Referencia Q3.12: 6434  | Signo seno: -1 | Signo coseno: 0

Ingrese el angulo en radianes: 6.2832
Entero Q3.12: 25736 | Caso especial: 0 o 2*pi | Indice de tabla: 0
Cuadrante: 0        | Referencia Q3.12: 0     | Signo seno: 0  | Signo coseno: 1
```

Figura 36 - Cálculo de cinco casos especiales

7 Estudio Comparativo para el Cálculo Entero del Índice

Validada la lógica de cuadrantes, restaba resolver cómo calcular el índice en Assembly sin emplear el punto flotante (float) utilizado provisoriamente en calcularIndice(). Para ello, se plantearon dos métodos matemáticos candidatos:

- **Opción A (Conversión explícita a grados)**: Transformar la referencia $Q3.12$ a radianes reales, multiplicarla por $\frac{180}{\pi}$ y redondear al grado entero más cercano:

$$índice_A = round\left(\frac{\theta_{ref}}{4096} \cdot \frac{180}{\pi}\right)$$

- **Opción B (Mapeo entero directo desde Q3.12)**: Calcular el índice mediante una proporción entera directa sobre el límite del primer cuadrante ($\pi/2 \approx 6434$ en $Q3.12$), sumando la mitad del divisor ($6434/2 = 3217$) en el numerador para efectuar el redondeo simétrico antes de la división entera:

$$índice_B = rou\left(\frac{\theta_{ref} \cdot 90+3217}{6434}\right)$$

Antes de decidir qué método llevar al ensamblador de la LC-3, se diseñó un banco de pruebas automatizado en C++ (prueba_indices.h y main_prueba_tabla.cpp) vinculado con la librería del grupo (Tabla.h) para evaluar el comportamiento de ambas fórmulas a lo largo de los **6435 valores posibles de referencia** (0 a 6434 inclusive).

Cabe destacar que en el prototipo de la Opción B se utilizó preventivamente int32_t para el numerador, dado que el producto máximo ($6434 \times 90 + 3217 = 582277$) supera el límite de un registro de 16 bits.

```
// Fragmento central del barrido exhaustivo sobre las 6435 referencias posibles (0 a 6434)
for (int32_t referencia = 0; referencia <= PI_sobre_2; referencia++) {
    double angulo = (double)referencia / 4096.0;

    // OPCION A: convertir a grados y redondear
    int indiceA = (int)round(angulo * 180.0 / PI_REAL);

    // OPCION B: calculo entero directo con redondeo (+ 6434 / 2)
    int32_t numerador = referencia * 90 + PI_sobre_2 / 2;
    int indiceB = numerador / PI_sobre_2;

    // Evaluacion contra la tabla Q1.14 real del proyecto y funciones ideales
    double senoReal = sin(angulo), cosenoReal = cos(angulo);
    double senoA = tablaQ14[indiceA] / 16384.0, cosenoA = tablaQ14[90 - indiceA] / 16384.0;
    double senoB = tablaQ14[indiceB] / 16384.0, cosenoB = tablaQ14[90 - indiceB] / 16384.0;
    // ... [Registro de discrepancias, errores maximos, promedios y RMS]
}
```

Figura 37 - Código referencial

7.1 Resultados del Barrido Exhaustivo, Auditoría del Caso Crítico (4754) y Criterio de Decisión

La ejecución del banco de pruebas sobre las 6435 referencias posibles (0 a 6434) vinculando los índices obtenidos con la tabla $Q1.14$ del proyecto (Tabla.h) arrojó tres hallazgos centrales que definieron el rumbo hacia Assembly:

- **Coincidencia del % 99.9845 y empate en errores máximos:** La Opción A y la Opción B seleccionaron exactamente el mismo índice de tabla en 6434 de las 6435 referencias. Al evaluar las salidas de la tabla $Q1.14$ frente a sin() y cos() ideales, ambas registraron el mismo error máximo global: 0.008690139 para el seno (en la referencia 250) y 0.008685693 para el coseno (en la referencia 6184).
- **Análisis de la discrepancia (θ_ref = 4754)**: La única diferencia entre ambos métodos aparece en el ángulo 66,500033°, prácticamente el límite de redondeo de 66,5°. Debido a las distintas aproximaciones utilizadas, la Opción A redondea a 67 (complementario 23), mientras que la Opción B redondea a 66 (complementario 24).

  Sin embargo, el impacto es mínimo: la Opción A es ligeramente más precisa en el seno y la Opción B en el coseno. Por lo tanto, ambos métodos presentan una precisión prácticamente equivalente.

- **Equivalencia estadística en el dominio completo (Promedio y RMS):** Al acumular el error absoluto y cuadrático medio (RMS) en las 6435 pruebas, las diferencias entre ambos métodos se redujeron al 10^{-9}, confirmando un empate técnico absoluto.

Métrica evaluada	Opción A	Opción B	Observación técnica
Caso puntual 4754 — Error seno	0,003471921	0,003486086	Índice A: 67 (sin = 0,920532) vs. B: 66 (sin = 0,913574)
Caso puntual 4754 — Error coseno	0,008001468	0,007989743	Comp. A: 23 (cos = 0,390747) vs. B: 24 (cos = 0,406738)
Global (6435 ref.) — Error promedio seno	0,0027773560	0,0027773582	Diferencia despreciable de $2{,}2\times10^{-9}$ a favor de A
Global (6435 ref.) — Error promedio coseno	0,0027773470	0,0027773452	Diferencia despreciable de $1{,}8\times10^{-9}$ a favor de B
Global (6435 ref.) — Error RMS seno / coseno	0,0035623678 / ..3562	0,0035623699 / ..3521	Empate práctico en dispersión cuadrática (10^{-9})

Figura 38 - Tabla Comparativa

Conclusión técnica de la etapa en C++: Los datos demuestran un **empate técnico absoluto en precisión** (10^{-9}). En consecuencia, se concluyó que la precisión matemática no inclina la balanza hacia ninguno de los dos métodos.

La elección definitiva entre la Opción A y la Opción B quedó sujeta exclusivamente al **costo de implementación en el Assembly original de la LC-3**, dado que por directivas de la cátedra solo se dispone inicialmente del repertorio nativo (ADD, AND, NOT, BR, cargas y saltos, sin instrucciones de multiplicación, ni división de otros grupos), el paso definitorio consiste en traducir ambas opciones a rutinas de 16 bits puras —resolviendo el riesgo de desbordamiento del producto intermedio $\theta_{ref} \times 90$ — y seleccionar aquella que demande menor cantidad de instrucciones ejecutadas y menor consumo de memoria.

7.2 Integración Modular e Interpolación Lineal Continua

Para incorporar el cálculo completo de seno y coseno sin comprometer las rutinas ya validadas en la primera versión de la biblioteca, se generó una copia evolutiva en c++. Se tomó la decisión de diseño de mantener intactos los identificadores de funciones, variables y constantes de la versión original y de redactar las nuevas rutinas en C++ con una sintaxis explícita y elemental (evitando constructores modernos complejos como static_cast, constexpr o bucles basados en rangos), garantizando una trazabilidad directa hacia las instrucciones de la arquitectura LC-3.

Conversión Continua a Grados, Monotonicidad y Caso Borde de 90°

A diferencia del prototipo preliminar —que redondeaba el ángulo al grado entero más cercano perdiendo la parte fraccionaria—, el paso a un esquema con interpolación exigió conservar los decimales del ángulo de referencia reducido. Para ello se incorporó la función convertir_referencia_a_grados(), la cual transforma la referencia $Q3.12$ a radianes reales y luego a grados sexagesimales:

```
// Convertir referencia Q3.12 a grados decimales
float convertir_referencia_a_grados(int16_t referencia_q3_12) {
    double radianes;
    double grados;

    radianes = (double)referencia_q3_12 / 4096.0;
    grados = radianes * 180.0 / 3.14159265358979323846;

    return (float)grados;
}
```

Figura 39 - Referencia a grados

Al someter esta función a un banco de pruebas aislado barriendo las **6435 referencias posibles** (0 a 6434), se evaluaron tres criterios numéricos:

- **Precisión de conversión:** El error absoluto máximo registrado fue de apenas 0.00000381° (en la referencia 6290), con un error absoluto promedio de 0.00000100°.
- **Comprobación de monotonicidad (0 errores):** Se verificó que la función sea estrictamente monótona creciente en todo el dominio (es decir, que ante cualquier incremento en la referencia $Q3.12$, el ángulo en grados nunca disminuya), asegurando que jamás se inviertan los índices vecinos al buscar en la tabla.
- **Detección del exceso en el extremo superior (6434 → 90.00025177°):** Al evaluar los casos particulares (0, 1, 2048, 4096, 5000, 6000 y 6434), se observó que el límite superior del primer cuadrante (6434) arrojaba 90.00025177°, superando levemente los 90°. Se demostró que este desvío **no constituye un defecto de la fórmula de conversión**, sino una consecuencia inherente a la cuantización de $\pi/2$ en formato $Q3.12$ ($\frac{\pi}{2} \times 4096 \approx 6433.98$, redondeado al entero 6434).

Implementación y Validación Exhaustiva de la Interpolación Lineal

Para resolver tanto la aproximación de ángulos intermedios como la protección de los extremos de memoria, se implementó en Tabla.h la función interpolarQ1_14(). La rutina incluye en su encabezado dos guardas de contorno que saturan cualquier entrada $\leq 0^{\circ}$ en tabla[0] y cualquier entrada $\geq 90^{\circ}$ en tabla[90], neutralizando el exceso de 90.00025° sin añadir complejidad aritmética.

Para cualquier ángulo intermedio (por ejemplo, 8.8°), la función separa la parte entera como índice inferior ($i = 8$), define el vecino consecutivo como índice superior ($i + 1 = 9$), extrae la distancia fraccionaria ($t = 8.8 - 8 = 0.8$) y calcula el aporte proporcional entre ambas muestras de la tabla $Q1.14$:

$$f(\theta) = tabla[i] + t \cdot (tabla[i + 1] - tabla[i])$$

```
// Interpolar un valor de seno en formato Q1.14
int16_t interpolarQ1_14(int16_t tabla[], float grados) {
    int indice_inferior;
    int indice_superior;
    float fraccion;
    float valor_inferior;
    float valor_superior;
    float resultado;

    // Controlar los limites de la tabla (evita desbordamiento en 6434 -> 90.00025°)
    if (grados <= 0) {
        return (int16_t)tabla[0];
    }
    if (grados >= 90) {
        return (int16_t)tabla[90];
    }

    // Obtener los indices vecinos y la parte fraccionaria
    indice_inferior = (int)grados;
    indice_superior = indice_inferior + 1;
    fraccion = grados - indice_inferior;

    // Obtener los valores vecinos e interpolar linealmente
    valor_inferior = tabla[indice_inferior];
    valor_superior = tabla[indice_superior];
    resultado = valor_inferior + fraccion * (valor_superior - valor_inferior);

    return (int16_t)(resultado + 0.5);
}
```

Figura 40 - Validación e interpolación

Para auditar la ganancia de precisión, se ejecutó el programa **Prueba_interpolacion.cpp** utilizando las funciones oficiales de inicialización de la librería (iniciar_tabla() y copiar_y_convertirQ1_14()) a lo largo de las 6435 referencias, obteniendo el coseno mediante la identidad de cofunción ($grados_coseno = 90.0 - grados$) sobre la misma tabla única:

Función Evaluada (6435 ref.)	Error Absoluto Máximo	Referencia del Peor Caso	Error Absoluto Promedio	Error Cuadrático Medio (RMS)
Seno Interpolado (Q1.14)	0.0000800950	5113	0.0000232687	0.0000287758
Coseno por Cofunción (Q1.14)	0.0000786832	1321	0.0000233238	0.0000287045

Figura 41 - Tabla Seno Interpolado y Coseno por Cofuncion

Los resultados experimentales confirmaron la solidez del modelo: al incorporar la interpolación lineal entre muestras de 1° (cuya cota teórica de truncamiento puro es $E_{\max} \leq \frac{h^2}{8} \approx 0.0000381$), el error absoluto máximo se redujo más de 100 veces respecto al esquema sin interpolación (bajando de ≈ 0.00869 a ≈ 0.000080, equivalente a apenas ≈ 1.3 unidades del escalón mínimo de cuantización en $Q1.14$, que es $1/16384 \approx 0.000061$).

7.3 Función (calcular_seno_coseno_Q1_14) y Unificación de Casos Especiales

El cierre de la librería en C++ consistió en acoplar todas las etapas dentro de la rutina principal calcular_seno_coseno_Q1_14(). Durante su diseño se estableció una distinción arquitectónica clave entre los **ángulos pertenecientes a un cuadrante (cuadrante = 1, 2, 3 o 4)** y los **casos especiales sobre los ejes cartesianos (cuadrante == 0)**:

- **Camino de Casos Especiales (cuadrante == 0):** En los ángulos situados exactamente sobre los ejes ($0^{\circ}, 90^{\circ}, 180^{\circ}, 270^{\circ}$ y 360°), el ángulo de referencia sigue existiendo geométricamente (por ejemplo, tanto 0° como 180° tienen referencia 0°), pero consultar la tabla resulta innecesario. Dado que determinar_cuadrante() asigna en signo_seno y signo_cos los estados exactos de la circunferencia trigonométrica (0, +1 o -1), en este camino dichos valores no actúan como multiplicadores de tabla, sino que determinan directamente el resultado final escalado por la constante UNO (16384 en $Q1.14$). Esto permite resolver los cinco bordes con una única expresión sin distinguir manualmente de qué eje se trata.
- **Camino de Cuadrantes Normales (cuadrante > 0):** Se convierte la referencia $Q3.12$ a grados, se interpola el seno en la tabla, se aplica la cofunción ($90^{\circ} - \theta_{ref}$) para interpolar el coseno en la misma estructura de datos y, por último, se multiplican ambas magnitudes positivas por sus respectivos factores de signo (+1 o -1).

```
// Calcular seno y coseno finales en Q1.14
void calcular_seno_coseno_Q1_14(int16_t tabla[], float angulo, int16_t &resultado_seno, int16_t &resultado_coseno){
    int16_t referencia;
    int cuadrante;
    int signo_seno, signo_cos;
    int16_t seno_referencia, coseno_referencia;
    float grados_referencia, grados_coseno;

    // 1. Determinar cuadrante, referencia y signos
    determinar_cuadrante(angulo, referencia, cuadrante, signo_seno, signo_cos);

    // 2. Resolver los casos especiales directamente sin consultar la tabla
    if (cuadrante == 0) {
        resultado_seno = signo_seno * UNO;
        resultado_coseno = signo_cos * UNO;
    }
    // 3. Camino normal: conversion, interpolacion de seno/coseno y signos
    else {
        grados_referencia = convertir_referencia_a_grados(referencia);
        seno_referencia = interpolarQ1_14(tabla, grados_referencia);

        grados_coseno = 90.0f - grados_referencia;
        coseno_referencia = interpolarQ1_14(tabla, grados_coseno);

        resultado_seno = seno_referencia * signo_seno;
        resultado_coseno = coseno_referencia * signo_cos;
    }
}
```

Figura 42 - Cálculo seno - coseno

7.4 Prueba Integral del Ciclo Completo (0 a 2π)

Con las rutinas **convertir_referencia_a_grados()**, **interpolarQ1_14()**, **determinar_cuadrante()** y **calcular_seno_coseno_Q1_14()** integradas en Tabla.h, se ejecutó la Prueba Integral (**Prueba_integral.cpp**) barriendo la totalidad de los 25737 valores enteros representables en formato $Q3.12$ a lo largo del ciclo completo $[0, 2\pi]$ (desde 0 hasta DOSxPI = 25736 inclusive) y contrastando cada salida frente a sin() y cos() de <cmath>:

Métrica Evaluada (Ciclo Completo 0 a 2π)	Seno (Q1.14)	Coseno (Q1.14)	Interpretación Técnica
Entradas Q3.12 evaluadas	2573725737	2573725737	Barrido exhaustivo de los 4 cuadrantes y 5 ejes
Error absoluto máximo	0.0000829186	0.0000877155	Peor caso en ref. 17981 (sin) y 21846 (cos)
Error absoluto promedio	0.0000231690	0.0000252635	Inferior a 0.42 unidades de precisión Q1.14
Error cuadrático medio (RMS)	0.0000285235	0.0000309332	Consistente con el promedio, sin picos anómalos

Figura 43 - Tabla de Pruebas

El análisis de estos resultados permitió validar tres aspectos clave del diseño antes de pasar al ensamblador:

- **Conservación del orden de magnitud del error en los 4 cuadrantes**: Mientras que en la prueba aislada del primer cuadrante el error máximo fue de 0.0000800950 (sin) y 0.0000786832 (cos), al extender el dominio a toda la circunferencia el peor desvío global apenas ascendió a 0.0000877155 (registrado en el coseno para la referencia 21846). Dado que un escalón de cuantización en $Q1.14$ equivale a $1/16384 \approx 0.000061035$, el error máximo en todo el ciclo representa apenas 1.44 unidades de punto fijo, confirmando que la reducción por simetrías y la propiedad de cofunción no degradan la precisión.
- **Exactitud absoluta en los ejes cartesianos (0 fallos)**: Los cinco casos especiales ($0^\circ, 90^\circ, 180^\circ, 270^\circ$ y 360°) devolvieron sin excepción los enteros extremos exactos (0, 16384 y -16384) mediante la vía directa (cuadrante == 0).

Evaluación de printf() frente a cout

Para evitar que en la consola de demostración un ángulo ingresado como 3.6 se imprimiera con el residuo de punto flotante (3.59999990) al usar fixed, el equipo propuso inicialmente incorporar la función printf() de la librería <stdio.h> heredada de C, aprovechando sus especificadores de formato para truncar visualmente la cantidad de decimales sin tocar el valor almacenado en memoria.

Se descartó el uso de printf() porque, para obtener una salida adaptativa sin ceros innecesarios, sería necesario utilizar %g. En cambio, C++ permite conseguir el mismo resultado de forma nativa mediante cout << defaultfloat << setprecision(7), mostrando hasta siete cifras significativas y eliminando los ceros residuales y el ruido de representación binaria. Así, se mantuvo <iostream> como único estándar de entrada/salida del proyecto, aclarando que estos métodos sólo modifican la visualización y no la precisión interna de los cálculos.

8 Planificación Arquitectónica para la Traducción a Assembly LC-3

Con la biblioteca Tabla.h verificada, se establecieron las siguientes decisiones para trasladar el algoritmo a Assembly LC-3:

- Convención de registros: Se reservaron R0 a R5 para operaciones, R6 para la pila y R7 para retornos de subrutinas. Se definió R0 como entrada (ángulo Q3.12), R1 como salida del seno y R2 como salida del coseno (ambos Q1.14).
- Administración de memoria: Debido a la disponibilidad limitada de registros, se decidió utilizar variables en memoria mediante .BLKW 1 para almacenar datos persistentes como signos, referencias e índices de tabla, accediendo a ellos con LD/ST o LDR/STR.
- Control de flujo mediante NZP: Para identificar los casos especiales sin reservar memoria adicional, se mantiene temporalmente el cuadrante en R3 y se utiliza ADD R3, R3, #0 para actualizar los códigos de condición antes de ejecutar BRz CASO_ESPECIAL.
- Constantes y operaciones aritméticas: Se definieron constantes Q1.14 mediante .FILL: 0, 16384 y -16384. Además, se verificó en el simulador la carga de la referencia crítica 4754 (x1292) y el funcionamiento de primitivas de multiplicación por sumas sucesivas y división por restas sucesivas.

Objetivo: establecer una arquitectura eficiente en registros, memoria y operaciones, preparando la implementación completa y la posterior medición de su costo en instrucciones LC-3

8.1 Entorno de Pruebas y Estrategia de Desarrollo en Assembly

Para la traducción a Assembly se utilizó el ensamblador online con depurador integrado desarrollado por otro grupo de la cátedra. Todo el código del módulo debía ejecutarse correctamente en este entorno, ya que posteriormente se incorpora mediante directivas **.EXTERN** a los códigos de los demás grupos para unificar la nueva ISA.

Para dejar el proceso bien documentado y poder aislar cualquier error, se adoptó una estrategia de **mini pruebas independientes**, avanzando de lo más simple a lo más complejo. Cada prueba se guardó en un archivo .asm propio, con un único objetivo y un resultado esperado definido de antemano. Previamente se modificó la tabla del prototipo de C++ a **int16_t**, verificando que el programa siguiera funcionando sin diferencias, y se copiaron al ensamblador las 91 directivas **.FILL** generadas por el programa, que conforman la etiqueta TABLA_SENO.

8.2 Prueba 01: Carga de un Elemento de la Tabla

El primer objetivo fue comprobar que el ensamblador acepta la tabla y que es posible recuperar un valor de ella. Se eligió el índice 30 porque su valor en Q1.14 (8192, equivalente a 0,5) es fácil de reconocer:

```
LEA R1, TABLA_SENO    ; R1 = direccion inicial de la tabla
        LDR R0, R1, #30       ; R0 = TABLA_SENO[30]
        TRAP x25
```

Figura 44 - Prueba 01

LEA carga en R1 la dirección de inicio de la tabla y LDR lee la palabra ubicada 30 posiciones después. Como cada .FILL ocupa exactamente una palabra, el desplazamiento coincide con el índice.

En el depurador se obtuvo **R0 = x2000 (8192)**, el valor esperado para el seno de 30°. Además, **R1 = x3003** (la tabla comienza justo después de las tres instrucciones del programa, que ocupan x3000 a x3002), por lo que el elemento leído se encuentra en x3003 + 30 = x3021.

8.3 Prueba 02: Acceso Dinámico a la Tabla

En el algoritmo real el índice no estará escrito en la instrucción, sino que se calculará durante la ejecución. Sin embargo, la instrucción LDR de la LC-3 admite únicamente un desplazamiento inmediato de 6 bits y no otro registro como índice. Por lo tanto, la dirección del elemento debe calcularse previamente con una suma:

```
LEA R1, TABLA_SENO    ; R1 = direccion inicial de la tabla
     AND R2, R2, #0
     ADD R2, R2, #15
     ADD R2, R2, #15       ; R2 = 30 (indice)
     ADD R3, R1, R2        ; R3 = direccion de TABLA_SENO[R2]
     LDR R0, R3, #0        ; R0 = valor de la tabla
     TRAP x25
```

Figura 45 - Prueba 02

El índice se construyó con dos sumas únicamente para probar el mecanismo. En la versión final se asume que el índice llega en un registro y que ya se encuentra dentro del rango válido (0 a 90), tal como se acordó con el grupo coordinador de la interfaz de entrada.

En el depurador se obtuvo **R0 = x2000 (8192)**, **R2 = x001E (30)**, **R1 = x3007** y **R3 = x3025**, coincidiendo con el cálculo x3007 + 30 = x3025. La rutina de acceso a la tabla queda entonces reducida a dos instrucciones (ADD y LDR), que constituyen el núcleo de lo que se integrará al módulo final.

8.4 Decisión de Diseño: Tabla Fija y Variables Temporales

Tras validar el acceso, la IA propuso como tercera prueba modificar posiciones de la tabla en tiempo de ejecución con STR para recuperarlas luego con LDR. Esta sugerencia se **descartó**: los valores de TABLA_SENO son constantes del módulo y no deben modificarse en ningún punto del programa. En su lugar, se planteó reservar espacios de memoria independientes para los datos intermedios que el algoritmo necesita, y comprobar que pueden escribirse y leerse sin afectar a la tabla.

Se definieron seis variables, cada una con su propia directiva **.BLKW #1** y una etiqueta propia (en lugar de un único vector), para que el código completo resulte claro al momento de leerlo y depurarlo:

Variable	Contenido	Formato
SIGNO_SENO	Signo del seno (-1, 0 o 1)	Entero con signo
SIGNO_COSENO	Signo del coseno (-1, 0 o 1)	Entero con signo
ANGULO_REFERENCIA	Ángulo reducido al primer cuadrante	Q3.12
RESULTADO_SENO	Resultado final del seno	Q1.14
RESULTADO_COSENO	Resultado final del coseno	Q1.14
REFERENCIA_GRADOS	Ángulo de referencia en grados	Q7.8

Figura 46 - Variables definidas

Se aclara que el ángulo recibido y el número de cuadrante no se reservaron en esta etapa. El ángulo original llegará en un registro y el cuadrante se mantendrá en un registro (R3), dado que ADD actualiza los códigos de condición (N, Z, P) y permite decidir con un BR si se trata de un caso especial o de un cuadrante normal, tal como se planificó en la sección 8.

8.5 Prueba 03: Reserva y Verificación de Variables Temporales

La prueba consiste en escribir un valor en cada variable con STR y recuperarlo inmediatamente con LDR. Los valores son deliberadamente pequeños y no representan todavía datos reales del algoritmo:

```
LEA R1, SIGNO_SENO
AND R0, R0, #0
ADD R0, R0, #-1
STR R0, R1, #0        ; SIGNO_SENO = -1
LDR R2, R1, #0        ; se recupera el valor
...                   ; (idem para las 6 variables)
```

Figura 47 - Prueba 03

Variable	Valor escrito	Hexadecimal
SIGNO_SENO	-1	xFFFF
SIGNO_COSENO	1	x0001
ANGULO_REFERENCIA	7	x0007
RESULTADO_SENO	11	x000B
RESULTADO_COSENO	-13	xFFF3
REFERENCIA_GRADOS	9	x0009

Figura 48 - Variables y Valores

Las seis posiciones de memoria conservaron el valor esperado. Además, como cada LDR recuperó correctamente el dato escrito por su STR, quedaron validadas ambas instrucciones: lo que se escribió en memoria es lo que luego se leyó.

8.6 Formato de la Salida: la Tabla ya Entrega Q1.14

Al planificar la función de conversión de la salida, se advirtió que **no era necesaria** ya que los valores de TABLA_SENO ya están almacenados en Q1.14, por lo que un valor leído de la tabla (por ejemplo 8192 para sen(30°) = 0,5) ya está listo para ser entregado. Multiplicarlo nuevamente por 16384, como hacía la conversión en C++, habría sido un error.

Asimismo, se aclaró que el resultado se entrega como **entero de 16 bits escalado**, no como una cadena de bits: el registro ya contiene el patrón binario, y decimal, hexadecimal y binario son tres representaciones del mismo contenido. El módulo de la interfaz interpreta esos 16 bits como Q1.14 y, si necesita mostrar el valor decimal, divide por 16384.

Resultado matemático	Valor Q1.14	Registro
0	0	x0000
0,5	8192	x2000
0,7071	11585	x2D41
1	16384	x4000
-1	-16384	xC000

Figura 49 - Formatos de Salida

8.7 Prueba 04: Determinación del Cuadrante

Como la LC-3 no posee instrucciones de comparación ni de resta, cada límite se compara en cascada, de forma análoga a lo planteado en la sección 6.5: se carga la constante, se obtiene su negativo con NOT y ADD #1, se suma al ángulo original (que permanece intacto en R0) y se salta según el resultado:

```
LDR R3, R4, #0        ; R3 = PI/2
NOT R3, R3
ADD R3, R3, #1        ; R3 = -PI/2
ADD R2, R0, R3        ; R2 = ANGULO - PI/2
BRn CUADRANTE_1       ; Si es negativo, cuadrante 1
```

Figura 50 - Prueba 04

El procedimiento se repite con π y 3π/2, y el cuadrante 4 se asigna por descarte. Las cuatro constantes (PI_SOBRE_2, PI, TRES_PI_SOBRE_2 y DOS_PI, en Q3.12) se declararon con .FILL en posiciones contiguas, de modo que un único LEA apunta a la primera y las demás se leen con LDR y desplazamientos 0, 1, 2 y 3. Como LEA y LD/ST usan un desplazamiento de 9 bits relativo al PC (±256 palabras), la tabla de 91 entradas deja margen suficiente para ubicar constantes y variables en cualquier posición del programa.

Se aplicaron además tres ajustes para dejar el código más claro y compatible con el entorno:

- Inicialización de R1 al comienzo (AND R1, R1, #0), de modo que cada asignación de cuadrante sea un único ADD.
- Etiquetas COMPROBAR_CUADRANTE_1 a _4, para que el flujo se parezca al de C++ y facilite la integración posterior.
- Eliminación de la etiqueta CONSTANTES: el ensamblador del otro grupo no acepta dos etiquetas consecutivas, por lo que en las pruebas siguientes siempre debe haber una instrucción o directiva entre etiquetas.

Los ángulos de prueba propuestos por la IA traían valores aproximados en grados con pequeñas diferencias, por lo que se recalcularon: 5000 ≈ 69,94° (cuadrante 1), 10000 ≈ 139,88° (cuadrante 2), 15000 ≈ 209,82° (cuadrante 3) y 22000 ≈ 307,74° (cuadrante 4; la IA indicaba 309,17°). El cuadrante resultante coincidía en todos los casos, pero el error se registró como ejemplo de la necesidad de verificar los cálculos del modelo. Al ejecutar el programa con ángulos de los cuatro cuadrantes, R1 resultó 1, 2, 3 y 4 respectivamente.

8.8 Prueba 05: Detección de Casos Especiales

Con la clasificación de cuadrantes validada en la Prueba 04, se amplió el programa para que identifique primero los cinco ángulos que caen exactamente sobre los ejes: 0, π/2, π, 3π/2 y 2π (0, 6434, 12868, 19302 y 25736 en Q3.12). Estas comprobaciones deben ir **antes** de la clasificación de cuadrantes: como esta usa BRn, que solo detecta valores estrictamente negativos, un ángulo igual a π/2, π o 3π/2 daría resta cero, no saltaría y se clasificaría por error en el cuadrante siguiente.

Se definió la convención de salida de R1: **0** indica caso especial y **1, 2, 3 o 4** indica el cuadrante normal. Como R1 se inicializa en 0 al comienzo del programa, la rama CASO_ESPECIAL no necesita escribir nada en él. Cada comprobación repite el mecanismo ya validado: se carga la constante, se obtiene su negativo con NOT y ADD #1, se suma al ángulo original (que permanece intacto en R0) y se salta con BRz si el resultado es cero.

```
COMPROBAR_90
        LDR R3, R4, #0  ; R3 = PI/2
        NOT R3, R3
        ADD R3, R3, #1  ; R3 = -PI/2
        ADD R2, R0, R3  ; R2 = ANGULO - PI/2
        BRz CASO_ESPECIAL ; SI DA 0, CASO ESPECIAL 90 GRADOS
```

Figura 51 - Prueba 05

El mismo bloque se repite con los desplazamientos #1, #2 y #3 para π, 3π/2 y 2π, mientras que el caso de 0° se resuelve directamente sobre el ángulo cargado (ver sección 8.9).

Se ejecutó el programa con los cinco casos especiales, obteniendo R1 = 0 en todos, y con un ángulo de cada cuadrante normal, obteniendo R1 = 1, 2, 3 y 4 respectivamente. De esta forma quedó validada la clasificación completa.

8.9 Optimización: Aprovechamiento de los Códigos de Condición

En la versión propuesta por la IA, la comprobación del ángulo nulo incluía una instrucción adicional antes del salto (ADD R2, R0, #0) con el único fin de actualizar los códigos de condición. Se advirtió que esa instrucción es redundante: LDR, al cargar un valor en un registro, actualiza N, Z y P según el dato cargado, por lo que BRz puede utilizar directamente el resultado de la carga del ángulo.

```
        LEA R4, ANGULO_PRUEBA
        LDR R0, R4, #0  ; LDR actualiza N, Z y P según el ángulo

COMPROBAR_CERO
        BRz CASO_ESPECIAL
```

Figura 52 - Usos de los NZP

Según la tercera edición de Patt y Patel, LEA no actualiza los códigos de condición, ya que carga una dirección efectiva y no un dato. Se adoptó entonces la siguiente convención para el resto del desarrollo: siempre que sea posible aprovechar los códigos de condición dejados por una instrucción anterior, se evita agregar un AND o un ADD solo para actualizarlos, ahorrando un ciclo de instrucción.

8.10 Prueba 06: Ángulo de Referencia y Cofunción

Una vez determinado el cuadrante, el paso siguiente consistió en calcular el ángulo de referencia. En C++ la cofunción se calculaba como 90° menos la referencia en grados, pero en Assembly todavía no se realiza la conversión al índice de la tabla. Se aprovechó entonces que $90^{\circ} = \pi/2$, de modo que la cofunción se calcula directamente en Q3.12, obteniendo el mismo ángulo sin conversión previa:

$$\theta_{cof} = \frac{\pi}{2} - \theta_{ref}$$

El ángulo de referencia depende del cuadrante:

- Cuadrante I: $\theta_{ref} = \theta$
- Cuadrante II: $\theta_{ref} = \pi - \theta$
- Cuadrante III: $\theta_{ref} = \theta - \pi$
- Cuadrante IV: $\theta_{ref} = 2\pi - \theta$

La asignación de registros fue la siguiente: R0 = ángulo original (Q3.12), R1 = cuadrante, R2 = ángulo de referencia (índice del seno), R3 = ángulo de cofunción (índice del coseno) y R4 = dirección de las constantes. Como R1 se utiliza como registro temporal durante las comparaciones, cada rama de cuadrante lo reinicia con AND y le asigna su valor con un único ADD. Dado que la LC-3 no posee instrucción de resta, todas las restas se resuelven con complemento a 2 (NOT + ADD #1).

```
CUADRANTE_2
        AND R1, R1, #0
        ADD R1, R1, #2
        LDR R2, R4, #1  ; R2 = PI
        NOT R2, R2
        ADD R2, R2, #1  ; R2 = -PI
        ADD R2, R0, R2  ; REFERENCIA = ANGULO - PI
        NOT R2, R2
        ADD R2, R2, #1  ; REFERENCIA = PI - ANGULO
        BR CALCULAR_COFUNCION
```

Figura 53 - Calculo Ángulo de referencia (ej: Cuadrante II)

```
CALCULAR_COFUNCION
        LDR R3, R4, #0  ; R3 = PI/2
        NOT R3, R3
        ADD R3, R3, #1  ; R3 = -PI/2
        ADD R3, R2, R3  ; R3 = REFERENCIA - PI/2
        NOT R3, R3
        ADD R3, R3, #1  ; R3 = PI/2 - REFERENCIA
        BR FIN
```

Figura 54 - Cálculo del ángulo de cofunción

Para los casos especiales no se calculan referencia ni cofunción: se dejan R1, R2 y R3 en cero y se termina, ya que sus resultados se resolverán más adelante con los signos. Se mantuvieron todos los comentarios de las pruebas anteriores, con el fin de que el código final quede documentado sin tener que recurrir a las pruebas previas.

Se ejecutó el programa con un ángulo por cada cuadrante, con los siguientes resultados esperados y obtenidos:

Ángulo (Q3.12)	R1 (cuadrante)	R2 (referencia)	R3 (cofunción)
5000	1	5000	1434
10000	2	2868	3566
15000	3	2132	4302
22000	4	3736	2698

Figura 55 - Resultados de la Prueba 06

8.11 Prueba 07: Determinación y Almacenamiento de los Signos

Con el cuadrante, el ángulo de referencia y la cofunción ya validados, el paso siguiente consistió en determinar el signo del seno y del coseno. Se conservó la lógica de la Prueba 06 y se agregó únicamente la determinación y el almacenamiento de ambos signos, dejando para una prueba posterior su aplicación a los resultados de la tabla.

Se acordó guardar los signos como enteros comunes de 16 bits (+1, 0 y −1) en las variables **SIGNO_SENO** y **SIGNO_COSENO**, reservadas con **.BLKW #1** y sin escalar todavía a Q1.14. Como el ángulo de entrada ya no es necesario una vez calculadas la referencia y la cofunción, se reutilizaron **R0** (signo del seno) y **R4** (signo del coseno) como registros temporales, conservando R1, R2 y R3 con los resultados de la Prueba 06.

Los signos dependen del cuadrante: en el I ambos son positivos; en el II el seno es positivo y el coseno negativo; en el III ambos son negativos; y en el IV el seno es negativo y el coseno positivo. En los cinco casos especiales, el signo coincide con el valor exacto sobre el eje (0, 1 o −1), por lo que se asigna directamente y se dejan R1, R2 y R3 en cero. Todas las ramas convergen en un único bloque que guarda ambos signos:

```
SIGNOS_CUADRANTE_2
        AND R0, R0, #0
        ADD R0, R0, #1          ; Signo del seno = +1
        AND R4, R4, #0
        ADD R4, R4, #-1         ; Signo del coseno = -1
        BR GUARDAR_SIGNOS

GUARDAR_SIGNOS
        LEA R5, SIGNO_SENO
        STR R0, R5, #0          ; Guardar signo del seno
        LEA R5, SIGNO_COSENO
        STR R4, R5, #0          ; Guardar signo del coseno
```

Figura 56 - Signos del cuadrante y almacenamiento en memoria

Los valores definitivos son los almacenados en las dos variables de memoria, que consultará la prueba que los aplique a los resultados de la tabla.

Al ejecutar por primera vez el programa con distintos ángulos de prueba, los valores almacenados no cambiaban: **SIGNO_SENO** quedaba siempre en 1 y **SIGNO_COSENO** siempre en 0, incluso con 22000, que debía dar −1 y +1. Como PI_SOBRE_2 es la primera constante y no ANGULO_PRUEBA, R0 recibía siempre 6434 (π/2) sin importar el valor de prueba. La corrección consistió en usar un segundo registro como puntero al ángulo, manteniendo R4 apuntando a las constantes:

```
LEA R4, PI_SOBRE_2        ; Cargar dirección de las constantes
LEA R5, ANGULO_PRUEBA     ; Cargar dirección del ángulo de prueba
LDR R0, R5, #0            ; Cargar ángulo de entrada Q3.12
```

Figura 57 - Corrección aplicada

Con la corrección se ejecutaron los nueve casos (cinco especiales y cuatro normales), revisando los valores de SIGNO_SENO y SIGNO_COSENO en el panel de memoria una vez finalizada la ejecución. Todos coincidieron con los esperados:

Ángulo (Q3.12)	Caso	SIGNO_SENO	SIGNO_COSENO
0	Especial (0°)	0	+1
6434	Especial (90°)	+1	0
12868	Especial (180°)	0	−1
19302	Especial (270°)	−1	0
25736	Especial (360°)	0	+1
5000	Cuadrante I	+1	+1
10000	Cuadrante II	+1	−1
15000	Cuadrante III	−1	−1
22000	Cuadrante IV	−1	+1

Figura 58 - Tabla de resultados

En los cuatro casos normales también se conservaron los resultados de la Prueba 06 en R1, R2 y R3 (por ejemplo, para 22000: R1 = 4, R2 = 3736 y R3 = 2698). Cabe aclarar que, al finalizar la ejecución, R4 y R5 no conservan las direcciones de las constantes ni del ángulo de prueba, ya que se reutilizan como registros temporales (R4 guarda el signo del coseno y R5 la dirección de SIGNO_COSENO). Por eso la verificación se realiza sobre las dos posiciones de memoria y no sobre esos registros. De este modo quedó validada la determinación de signos, y las dos variables quedan disponibles para aplicarse sobre los resultados de la tabla.

8.12 Prueba 08 A: Conversión de la Referencia a Grados Q8.8 e Índice de la Tabla

Con el cuadrante, la referencia, la cofunción y los signos validados, el paso siguiente consistió en convertir el ángulo de referencia (Q3.12) en el índice entero de la tabla de 91 entradas. La Prueba 08 se dividió en fases: la Fase A, que se describe aquí y llega hasta el índice, y la Fase B, que utilizará ese índice para acceder a la tabla de seno. Se mantuvo la estrategia validada en C++ (conversión explícita a grados Q8.8 y redondeo al entero más cercano, como en (int)(grados + 0.5f)), descartando calcular el índice en un solo paso, ya que una prueba paralela con esa alternativa no funcionó:

$$G_{Q8.8} = \frac{R_{Q3.12} \times 58671}{16384}, \quad I = \left\lfloor \frac{G_{Q8.8} + 128}{256} \right\rfloor$$

El producto $R \times 58671$ no cabe en 16 bits (para 90°, $6434 \times 58671 = 377489214$ necesita 29 bits), por lo que se usó un acumulador de 32 bits en dos registros, R2 (palabra alta) y R1 (palabra baja). Como la LC-3 no tiene multiplicación, se utilizó la descomposición binaria $58671 = 2^{15} + 2^{14} + 2^{13} + 2^{10} + 2^{8} + 2^{5} + 2^{3} + 2^{2} + 2^{1} + 2^{0}$. Se recorren los bits del 15 al 0, en cada uno se duplica el acumulador y, si el bit vale 1, se suma la referencia R0. Ambas operaciones son subrutinas (JSR/RET) que transportan el acarreo entre palabras, ya que la LC-3 no tiene bandera de acarreo:

```
DOBLAR_ACUMULADOR
        ADD R3, R1, #0
        BRn DOBLAR_CON_ACARREO ; Bit 15 de R1 = 1: habrá acarreo hacia R2
        ADD R1, R1, R1
        ADD R2, R2, R2
        RET
DOBLAR_CON_ACARREO
        ADD R1, R1, R1
        ADD R2, R2, R2
        ADD R2, R2, #1         ; Transferir el acarreo a la palabra alta
        RET
```

Figura 59 - Acumulador

Dividir por 16384 equivale a desplazar el producto 14 bits a la derecha: $G_{Q8.8} = 4 \cdot R2 + bits\ 15\ y\ 14\ de\ R1$. Luego se suma 128 (medio grado) y se divide por 256 mediante restas sucesivas, de modo que cada resta exitosa incrementa el índice.

```
        AND R4, R4, #0
        ADD R3, R1, #0   ; Corrección: el signo de R1 es su bit 15
        BRzp BIT_15_CERO
        ADD R4, R4, #2   ; Bit 15 = 1: aporta 2
BIT_15_CERO
        ADD R3, R3, R3   ; Coloca el bit 14 en la posición 15
        BRzp BIT_14_CERO
        ADD R4, R4, #1   ; Bit 14 = 1: aporta 1
BIT_14_CERO
        ADD R2, R2, R2
        ADD R2, R2, R2   ; R2 = palabra alta x 4
        ADD R2, R2, R4   ; R2 = grados Q8.8
```

Figura 60 - Extracción de los grados Q8.8

Se ejecutó el programa con cinco referencias y los valores coincidieron con los calculados en Python (se almacenan en PRODUCTO_BAJO, PRODUCTO_ALTO, GRADOS_Q8_8 e INDICE_TABLA). Además, se comprobó en Python sobre los 6435 valores posibles (0 a 6434) que el índice obtenido coincide con redondear el resultado exacto de la fórmula:

Referencia (Q3.12)	Producto alto	Producto bajo	Grados Q8.8	Índice
0	0	x0000	0	0
2145 (~30°)	1920	x4ECF	7681	30
3217 (~45°)	2880	x039F	11520	45
4289 (~60°)	3839	xB86F	15358	60
6434 (90°)	5760	x073E	23040	90

Figura 61 - Resultados de la prueba

8.13 Prueba 08 B: Índices de Seno y Coseno a partir de la Cofunción

Con el índice del seno validado en la Fase A, la Fase B tuvo como objetivo obtener, en una misma ejecución, el índice del seno y el del coseno. La estrategia consiste en reutilizar, sin modificarla, la conversión de la Prueba 08 A (multiplicación por 58671, extracción de los grados Q8.8, redondeo y división por restas sucesivas), alimentándola primero con la referencia del seno y luego con la del coseno. Esta última no requiere ningún cálculo adicional: es la cofunción obtenida en la Prueba 06, que en el programa principal se encuentra en R3. De este modo, el índice del coseno se obtiene consultando la misma TABLA_SENO con el ángulo complementario.

Como el módulo se probó de forma independiente, ambas referencias se cargan desde memoria con **REFERENCIA_SENO_PRUEBA** y **REFERENCIA_COSENO_PRUEBA**; al integrarlo, estas cargas se reemplazarán por los valores que ya están en R2 y R3.

El primer cálculo, correspondiente al seno, es el de la Fase A. Su única diferencia es que el índice obtenido se guarda en la variable **INDICE_SENO** antes de reutilizar los registros:

```
; PRIMER CALCULO: INDICE DEL SENO
        LEA R5, REFERENCIA_SENO_PRUEBA
        LDR R0, R5, #0          ; R0 = referencia del seno (Q3.12)

        AND R1, R1, #0          ; acumulador de 32 bits R2:R1 en cero
        AND R2, R2, #0
        ...                     ; multiplicacion por 58671 (igual a 8A)
        ...                     ; conversion a Q8.8, +128 y division por 256
FIN_INDICE_SENO
        LEA R5, INDICE_SENO
        STR R4, R5, #0          ; se conserva el indice del seno en memoria
```

Figura 62 - Cálculo del índice del seno y almacenamiento en INDICE_SENO

El segundo cálculo carga la referencia del coseno en R0 y reinicia el acumulador, ya que de lo contrario se arrastrarían los valores del cálculo anterior. A partir de ahí repite exactamente el mismo procedimiento, con etiquetas propias para no duplicar nombres:

```
; SEGUNDO CALCULO: INDICE DEL COSENO
        LEA R5, REFERENCIA_COSENO_PRUEBA
        LDR R0, R5, #0          ; R0 = referencia del coseno (cofuncion)

        AND R1, R1, #0          ; reinicio del acumulador R2:R1
        AND R2, R2, #0
        ...                     ; misma multiplicacion por 58671
OBTENER_INDICE_COSENO
        ADD R3, R2, R5          ; R5 = -256
        BRn FIN_INDICE_COSENO
        ADD R2, R3, #0
        ADD R4, R4, #1          ; cada resta exitosa suma 1 al indice
        BR OBTENER_INDICE_COSENO
FIN_INDICE_COSENO
        LEA R5, INDICE_COSENO
        STR R4, R5, #0
```

Figura 63 - Cálculo del índice del coseno

Las subrutinas **DOBLAR_ACUMULADOR** y **SUMAR_REFERENCIA** se comparten entre ambos cálculos sin cambios. Para que el segundo cálculo no sobreescriba al primero, cada uno tiene sus propias variables. Al finalizar, ambos índices se cargan desde memoria en los registros de salida: R1 para el seno y R2 para el coseno.

```
        LEA R5, INDICE_SENO
        LDR R1, R5, #0          ; R1 = indice del seno

        LEA R5, INDICE_COSENO
        LDR R2, R5, #0          ; R2 = indice del coseno
        HALT
```

Figura 64 - Registros de Salida

Para la validación se emplearon cinco **pares de referencias complementarias**, es decir, que suman aproximadamente 90°. Probar en pares, en lugar de cada referencia por separado, permite verificar también que la relación de cofunción se mantiene.

Prueba	Referencia seno (Q3.12)	Referencia coseno (Q3.12)	Índice seno	Índice coseno
1	0	6434	0	90
2	2145	4289	30	60
3	3217	3217	45	45
4	4289	2145	60	30
5	6434	0	90	0

Figura 65 - Valores y resultados

Con esta prueba quedó validada la obtención de los dos índices a partir de las referencias del seno y del coseno, conservados en R1 y R2 y también en memoria. El procedimiento de conversión de la Fase A se reutilizó sin cambios, y la cofunción calculada en la Prueba 06 permitió resolver el coseno consultando la misma TABLA_SENO, sin agregar una segunda tabla.

8.14 Prueba 08 C: Consulta de los Valores en la Tabla

Con los índices del seno y del coseno disponibles, la Fase C tuvo como objetivo recuperar de TABLA_SENO los valores correspondientes y entregarlos en los registros de salida.

Se conservó todo el código de la Prueba 08 B y se reemplazó únicamente el bloque final de preparación de registros por la consulta a la tabla. Se mantuvo el criterio de acceso dinámico fijado en la sección anterior: **LEA** obtiene la dirección base de la tabla, **LDR** recupera el índice guardado en memoria, **ADD** calcula la dirección del elemento y un segundo **LDR** trae el valor Q1.14. Como se explicó en la Prueba 02, la suma es necesaria porque el desplazamiento de LDR es de solo 6 bits.

```
LEA R3, TABLA_SENO      ; R3 = direccion base de la tabla

LEA R5, INDICE_SENO
LDR R4, R5, #0          ; R4 = indice del seno
ADD R5, R3, R4          ; direccion = base + indice
LDR R1, R5, #0          ; R1 = valor del seno (Q1.14)
LEA R5, RESULTADO_SENO
STR R1, R5, #0
```

Figura 66 - Consulta dinámica del valor del seno

El valor del coseno se obtiene con el mismo procedimiento, usando **INDICE_COSENO** y dejando el resultado en R2:

```
LEA R5, INDICE_COSENO
LDR R4, R5, #0          ; R4 = indice del coseno
ADD R5, R3, R4          ; direccion = base + indice
LDR R2, R5, #0          ; R2 = valor del coseno (Q1.14)
LEA R5, RESULTADO_COSENO
STR R2, R5, #0
HALT
```

Figura 67 - Consulta dinámica del valor del coseno

Además de las variables **RESULTADO_SENO** y **RESULTADO_COSENO**, se incorporó al final del programa la tabla de 91 elementos, sin modificar ninguno de sus valores. Se reutilizaron los cinco pares de la Prueba 08 B y sus resultados , todos en Q1.14:

Prueba	Índice seno	Índice coseno	R1 esperado	R2 esperado
1	0	90	0	16384
2	30	60	8192	14189
3	45	45	11585	11585
4	60	30	14189	8192
5	90	0	16384	0

Figura 68 - Resultados esperados

En las cinco ejecuciones, R1 y R2 contuvieron los valores esperados.

Cabe aclarar que, en esta fase, los valores recuperados son siempre positivos, ya que la tabla sólo contiene el primer cuadrante. Los signos se aplican en la fase siguiente.

8.15 Prueba 08 D: Aplicación de los Signos

La Fase D toma los valores positivos de R1 y R2 y les aplica los signos determinados en la Prueba 07, almacenados en **SIGNO_SENO** y **SIGNO_COSENO**.

La regla es la siguiente: si el signo es +1, el valor se conserva, si es −1, se obtiene su opuesto mediante complemento a dos (**NOT** seguido de **ADD #1**), y si es 0 el resultado se fuerza a cero. Como el valor del signo se carga con LDR, el mismo ADD que lo actualiza permite decidir con BRz y BRn sin instrucciones adicionales:

```
        LEA R5, SIGNO_SENO
        LDR R3, R5, #0          ; R3 = signo del seno (+1, -1 o 0)
        ADD R3, R3, #0
        BRz SENO_CERO           ; signo 0: resultado = 0
        BRn SENO_NEGATIVO       ; signo -1: cambiar de signo
        BRnzp SENO_FINALIZADO   ; signo +1: se conserva el valor

SENO_NEGATIVO
        NOT R1, R1
        ADD R1, R1, #1          ; complemento a dos
        BRnzp SENO_FINALIZADO

SENO_CERO
        AND R1, R1, #0
SENO_FINALIZADO
```

Figura 69 - Aplicación del signo al seno

El coseno utiliza una estructura idéntica, con **SIGNO_COSENO** y el registro R2. Al finalizar, los resultados ya con su signo se guardan nuevamente en RESULTADO_SENO y RESULTADO_COSENO, quedando R1 y R2 como salidas definitivas del módulo.

Se verificaron los cuatro cuadrantes, que abarcan todas las combinaciones de signos posibles, y los cinco casos especiales:

Ángulo	Signo seno	Signo coseno	R1 esperado	R2 esperado
30°	+1	+1	8192	14189
120°	+1	−1	14189	−8192
210°	−1	−1	−8192	−14189
300°	−1	+1	−14189	8192
0°	0	+1	0	16384
90°	+1	0	16384	0
180°	0	−1	0	−16384
270°	−1	0	−16384	0
360°	0	+1	0	16384

Figura 70 - Casos de prueba de la Prueba 08 D

Los valores negativos se verifican en complemento a dos; por ejemplo, −8192 aparece como xE000 en el depurador.

8.16 Prueba 08 E: Integración con la Determinación de Cuadrantes y Entrada Única

Al intentar ejecutar la Prueba 08 D, el ensamblador informó **Label SIGNO_SENO doesn't exist**. La causa fue que los signos eran producidos por la Prueba 07, cuya lógica todavía no se había incorporado al programa, y no solo faltaban las dos variables sino todo el procedimiento que determina el cuadrante, la referencia, la cofunción y los signos. Este error derivó en la integración de las pruebas, que constituye la Fase E: el programa recibe un **único ángulo en Q3.12** y calcula por sí solo la referencia del seno, la referencia del coseno por cofunción, ambos índices, ambos valores y sus signos. En la versión final del módulo, ese ángulo llegará por registro.

La lógica de la Prueba 07 se insertó inmediatamente después de INICIO y antes del cálculo del índice del seno. Se descartaron su .ORIG, su etiqueta INICIO y su HALT final, y se realizaron tres ajustes para conectarla con las Fases B, C y D:

- **Casos especiales:** En la Prueba 07 solo se guardaban los signos; ahora cada caso especial deja además preparadas las referencias que consultará la tabla. Por ejemplo, para 0° y 180° la referencia del seno es 0 y la del coseno es π/2, mientras que para 90° y 270° ocurre lo contrario:

```
CASO_ESPECIAL_180
        AND R1, R1, #0
        AND R0, R0, #0          ; signo del seno = 0
        AND R4, R4, #0
        ADD R4, R4, #-1         ; signo del coseno = -1
        AND R2, R2, #0          ; referencia del seno = 0
        LEA R5, PI_SOBRE_2
        LDR R3, R5, #0          ; referencia del coseno = 90 grados
        BR GUARDAR_SIGNOS
```

Figura 71 - Caso especial 180° con sus referencia

- **Almacenamiento de referencias y signos.** La etiqueta GUARDAR_SIGNOS ahora guarda también las referencias calculadas (R2 para el seno y R3 para el coseno) en **REFERENCIA_SENO_PRUEBA** y **REFERENCIA_COSENO_PRUEBA**, que dejan de ser constantes y pasan a ser variables reservadas con .BLKW. De esta manera, las Fases B, C y D reciben sus entradas desde memoria sin cambios, y el salto final transfiere el control a una nueva etiqueta, CALCULO_SENO, ubicada al comienzo del primer cálculo, para que el flujo no entre en él antes de terminar de determinar los signos:

```
GUARDAR_SIGNOS
        LEA R5, REFERENCIA_SENO_PRUEBA
        STR R2, R5, #0          ; referencia del seno
        LEA R5, REFERENCIA_COSENO_PRUEBA
        STR R3, R5, #0          ; referencia del coseno (cofuncion)
        LEA R5, SIGNO_SENO
        STR R0, R5, #0
        LEA R5, SIGNO_COSENO
        STR R4, R5, #0
        BR CALCULO_SENO         ; continuar con los indices (Prueba 08 B)
```

Figura 72 - Almacenamiento de referencias y signos

- **Sección de datos:** Se reemplazaron las dos referencias de prueba por el ángulo de entrada **ANGULO_PRUEBA**, las cuatro constantes Q3.12 (π/2, π, 3π/2 y 2π) y las variables de referencias y signos.

```
ANGULO_PRUEBA            .FILL #6434     ; unica entrada del programa
PI_SOBRE_2               .FILL #6434
PI                       .FILL #12868
TRES_PI_SOBRE_2          .FILL #19302
DOS_PI                   .FILL #25736

REFERENCIA_SENO_PRUEBA   .BLKW #1
REFERENCIA_COSENO_PRUEBA .BLKW #1
SIGNO_SENO               .BLKW #1
SIGNO_COSENO             .BLKW #1
```

Figura 73 - Datos del programa integrado

El flujo completo queda entonces:

1. Lectura del ángulo
2. Identificación del caso especial o del cuadrante,
3. Cálculo de la referencia y de la cofunción,
4. Almacenamiento de signos,
5. Obtención de los índices (08 B),
6. Consulta de la tabla (08 C)
7. Aplicación de los signos (08 D).

Se comenzó con una prueba sencilla con 90°. Luego se repitió la validación con ángulos de los cuatro cuadrantes y los cinco casos especiales de la tabla anterior, y en todos los casos los valores obtenidos coincidieron con los esperados.

De esta forma quedó validado el cálculo completo de seno y coseno a partir de un único ángulo, con el seno en R1 y el coseno en R2, ambos en Q1.14 y con su signo correcto.

9 Conclusión y Trabajo Futuro

El desarrollo en Assembly LC-3 se llevó a cabo junto con la IA, avanzando con pruebas pequeñas e independientes, cada una con un resultado esperado definido de antemano. La IA propuso el código y la estructura de cada fase, mientras que la verificación en el depurador, la corrección de errores y las decisiones de diseño estuvieron a cargo del grupo. Aunque el asistente cometió errores en el camino (ángulos con grados mal calculados, una prueba que modificaba la tabla, una fase que descartaba el índice del seno y la omisión de la lógica de signos al integrar), el trabajo de revisar cada resultado permitió detectarlos y corregirlos, y se logró construir un módulo funcional: a partir de un único ángulo en Q3.12 entrega el seno en R1 y el coseno en R2, en Q1.14 y con su signo correcto, para los cuatro cuadrantes y los cinco casos especiales.

Como **trabajo futuro** queda implementar en Assembly la **interpolación lineal** ya validada en el prototipo de C++, de modo que el módulo también sea preciso, y luego medir el costo en instrucciones del módulo completo.

10 REFERENCIAS BIBLIOGRÁFICAS

[1] S. Harris and D. Harris, *Digital design and computer architecture: arm edition*: Morgan Kaufmann, 2015.

[2] Y. N. Patt and S. J. Patel, *Introduction to computing systems: from bits and gates to C and beyond*, 3rd ed.: McGraw-Hill Education, 2019.

[3] Cátedra de Arquitectura de Computadoras , *Ensamblador online con depurador integrado para LC-3*: desarrollado por otro grupo de la cátedra, Facultad de Ingeniería, UNAM.